IF69D-S11 e OPEET014-OPEL

Processamento Digital de Imagens

Aula 08 – Segmentação de Imagens

Universidade Tecnológica Federal do Paraná - UTFPR - Curitiba
Departamento Acadêmico de Eletrônica - DAELN
Curso de Engenharia Eletrônica
Prof. Gustavo B. Borba e Prof. Humberto R. Gamba

13.out.2026

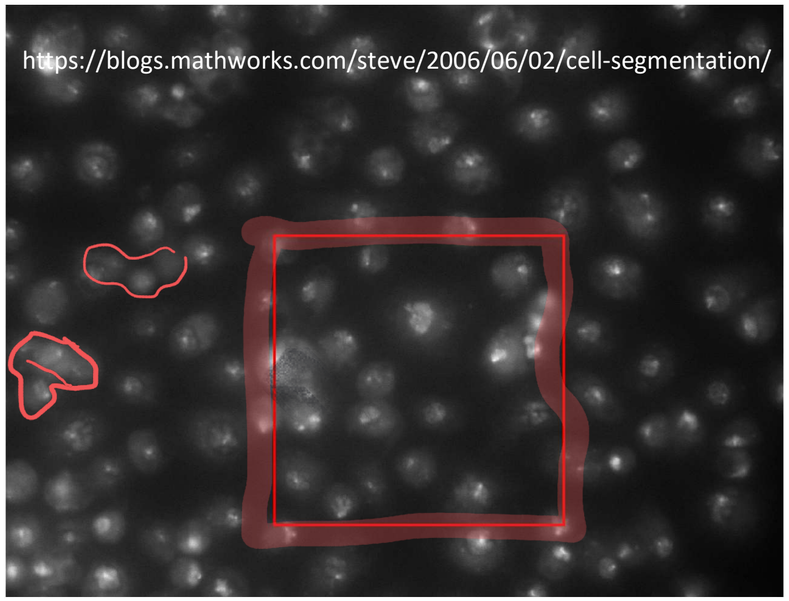
Image segmentation: **O que segmentação ?**

Objetivo da segmentação: separar automaticamente cada núcleo do fundo e dos demais.

Aplicações de segmentação: Separar automaticamente objetos em uma imagem, e avaliar o seu conteúdo, respondendo:

- Quantos objetos (células) existem na imagem?
- Como separar objetos adjacentes?
- Como definir automaticamente as fronteiras?

Imagem de núcleos celulares obtida por microscopia de fluorescência.



Núcleos celulares (microscopia de fluorescência)

Recorte (ROI): foco em uma região para análise

Facilitar a visualização e o entendimento das técnicas de segmentação.

```
clc, clear, close all
I = imread('https://blogs.mathworks.com/images/steve/60/nuclei.png');
figure, imshow(I,[]);
hold on
rectangle('Position',[465 400 500 500], 'EdgeColor','r', 'LineWidth',2);
I_cropped = I(400:900, 465:965);
figure, imshow(I_cropped,[]);
```

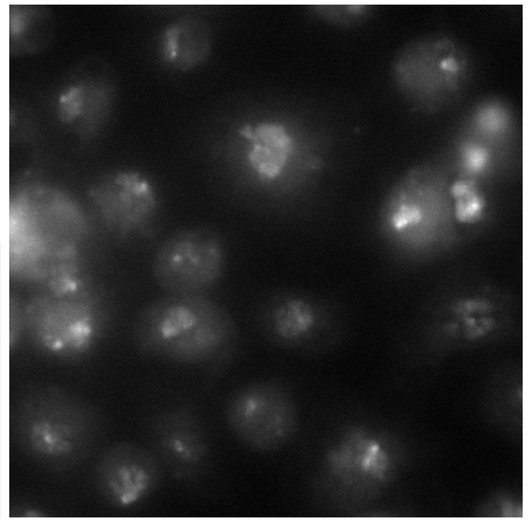
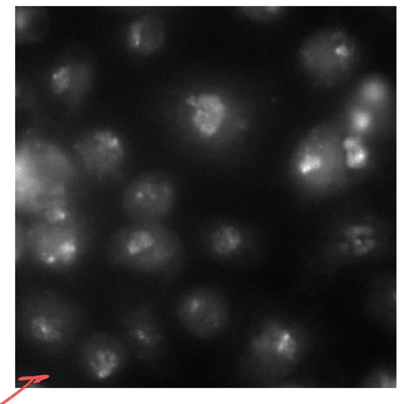
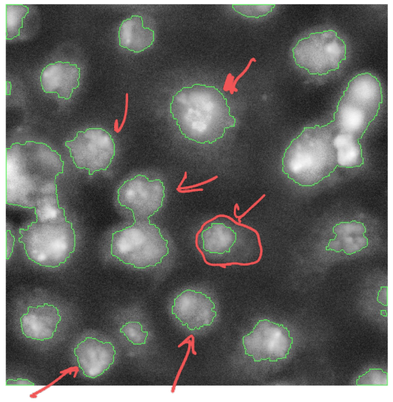


Image segmentation: **O que é segmentação?**

Objetivo: separar automaticamente objetos de interesse do fundo (e entre si).

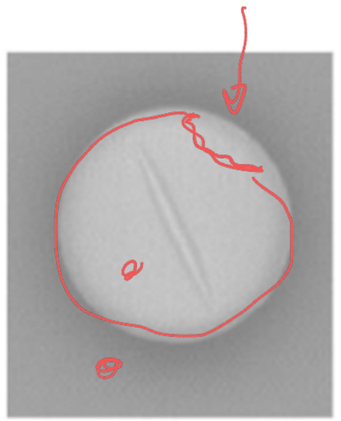


Núcleos celulares

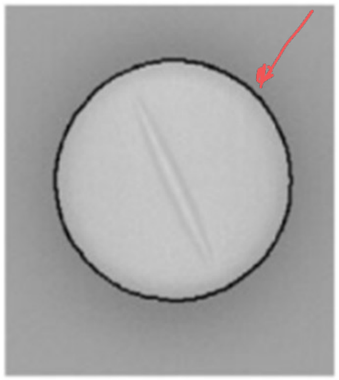


Segmentação (contorno)

https://blogs.mathworks.com/steve/2006/06/02/cell-segmentation/



Inspeção industrial



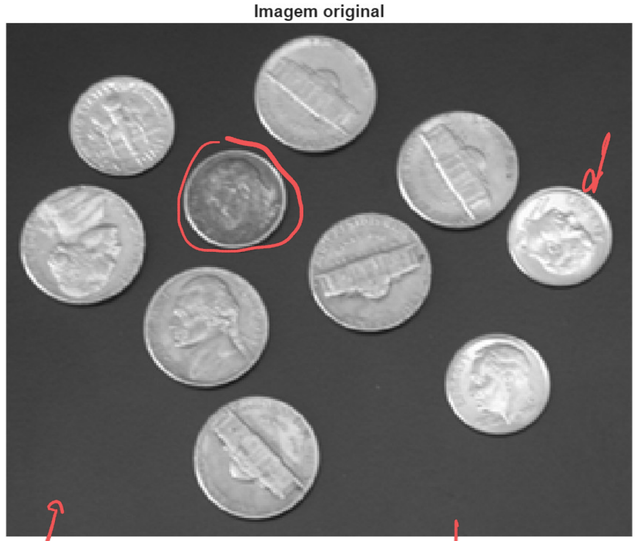
Contorno detectado

Miha Mozina, Dejan Tomazevic, Franjo Pernus, Bostjan Likar, Real-time image segmentation for visual inspection of pharmaceutical tablets, Machine Vision and Applications, v. 22, p. 145-156, 2011, DOI 10.1007/s00138-009-0218-7.

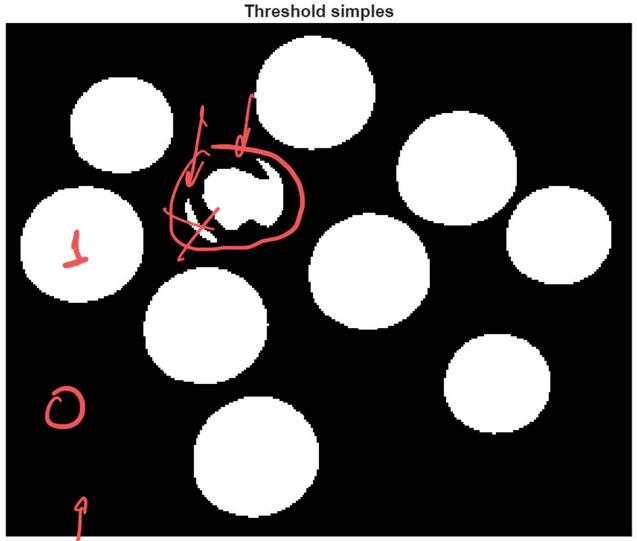
Image segmentation: **O que é segmentação?**

Exemplo de um pipeline de segmentação

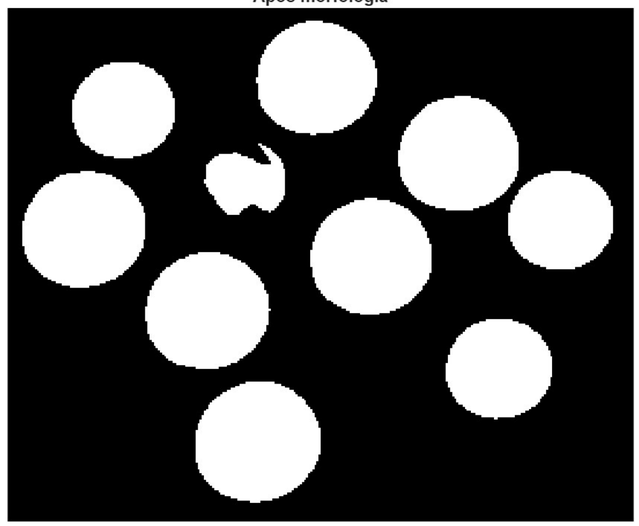
Imagem original



Threshold simples



Após morfologia



Contorno detectado

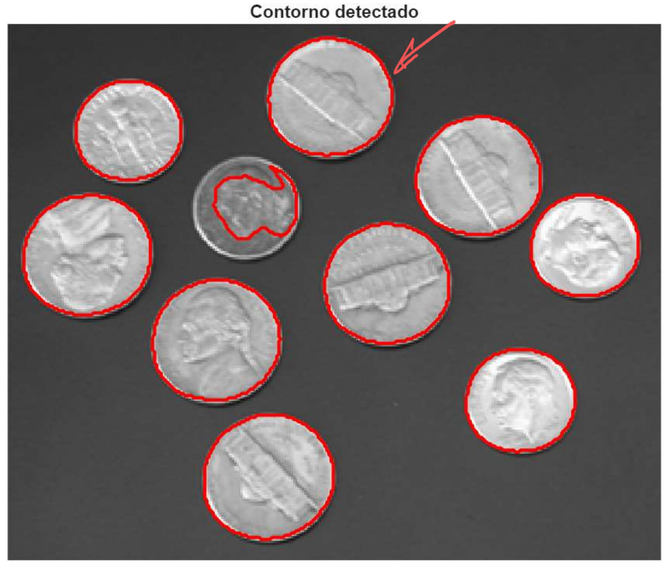


Image segmentation: **Limiariação global (thresholding)**

ideia: separar por intensidade, transformando uma imagem em binária:

$$g(x,y) = \begin{cases} 1 & f(x,y) > T \\ 0 & \text{caso contrário} \end{cases}$$

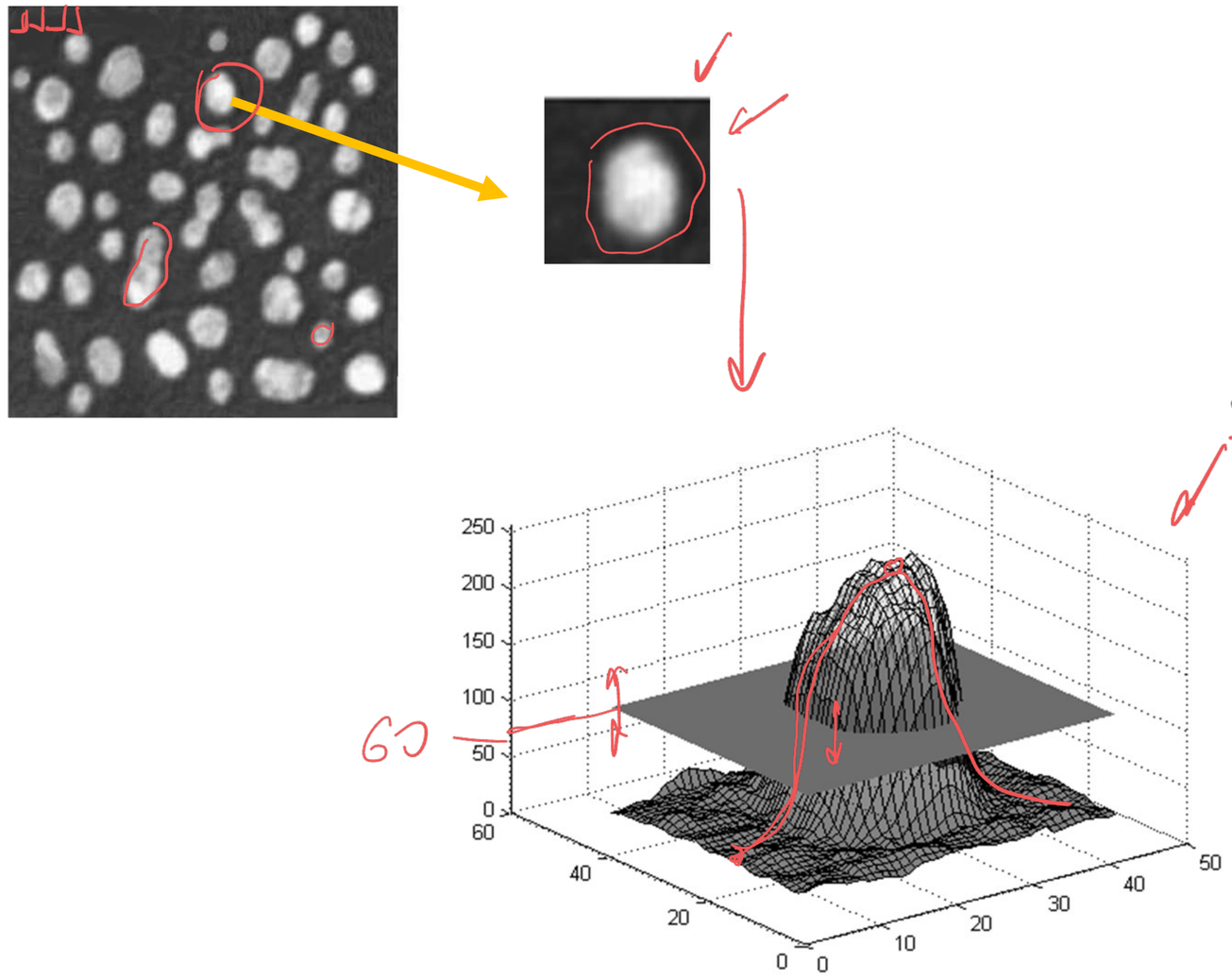


Imagem adaptada de: Geoff Dougherty, Digital Image Processing for Medical Applications, Cambridge University Press, 2009, Figure 9.11.
https://www.cambridge.org/br/academic/subjects/engineering/biomedical-engineering/digital-image-processing-medical-applications?format=HB

Image segmentation: **Limiariação global (thresholding)**

Qual separa melhor objeto e fundo?

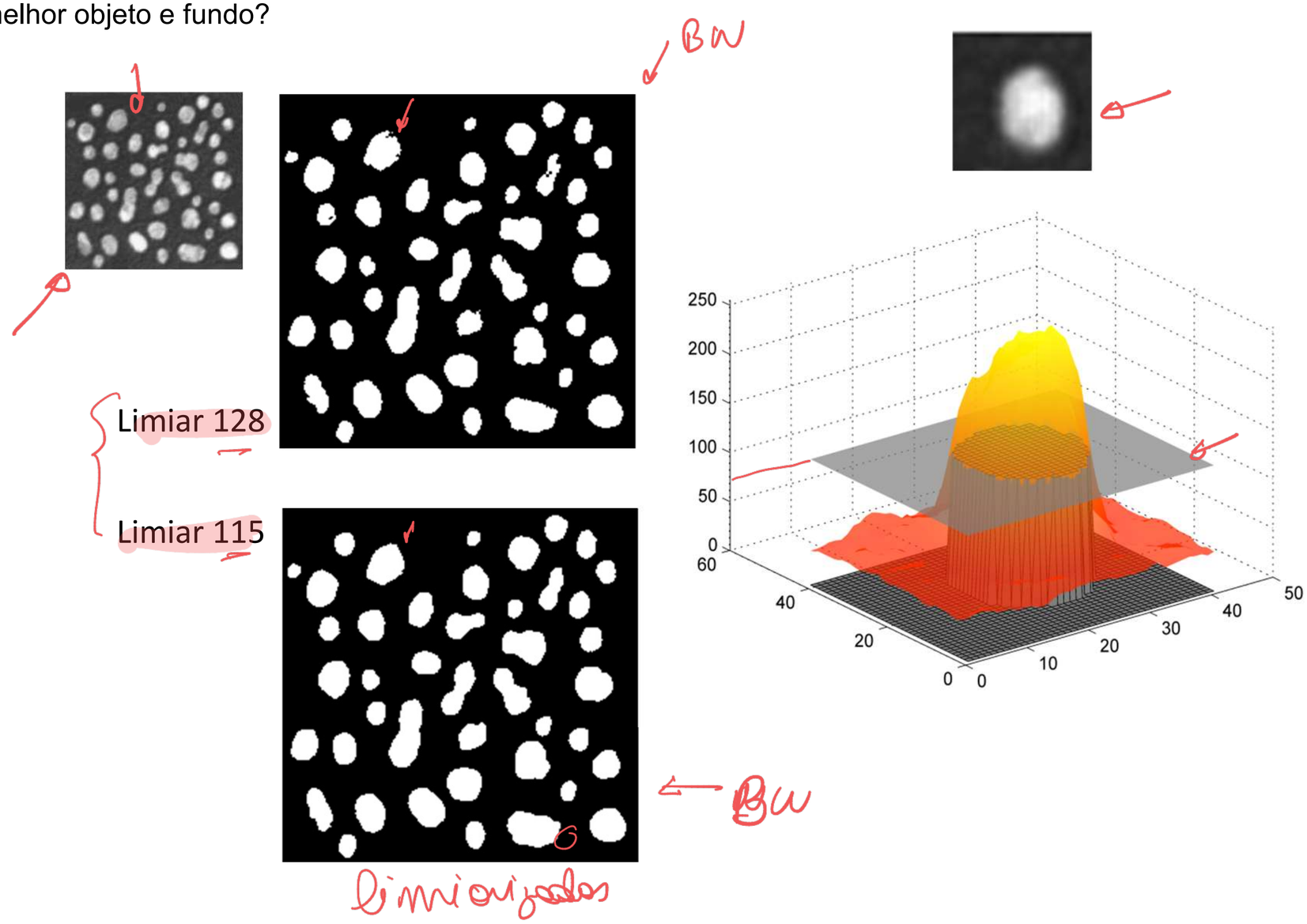
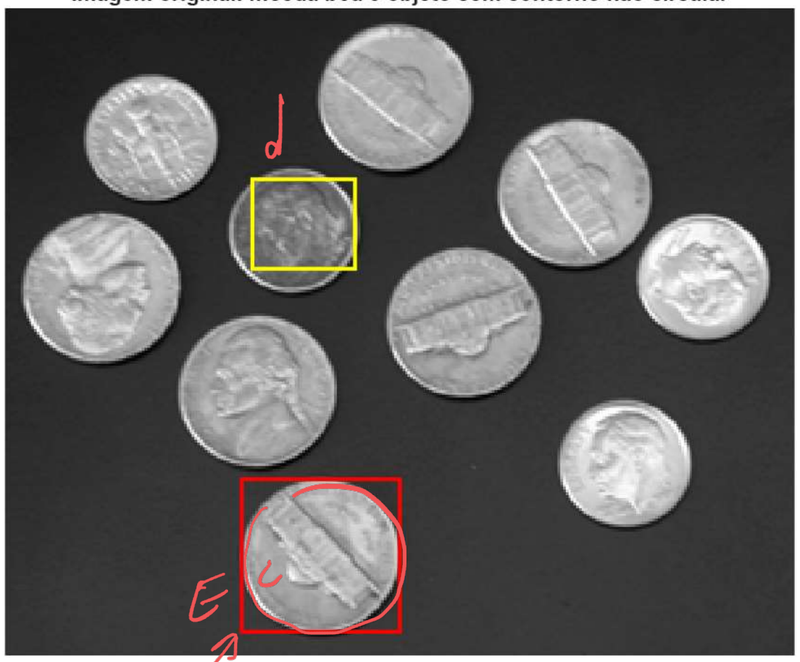
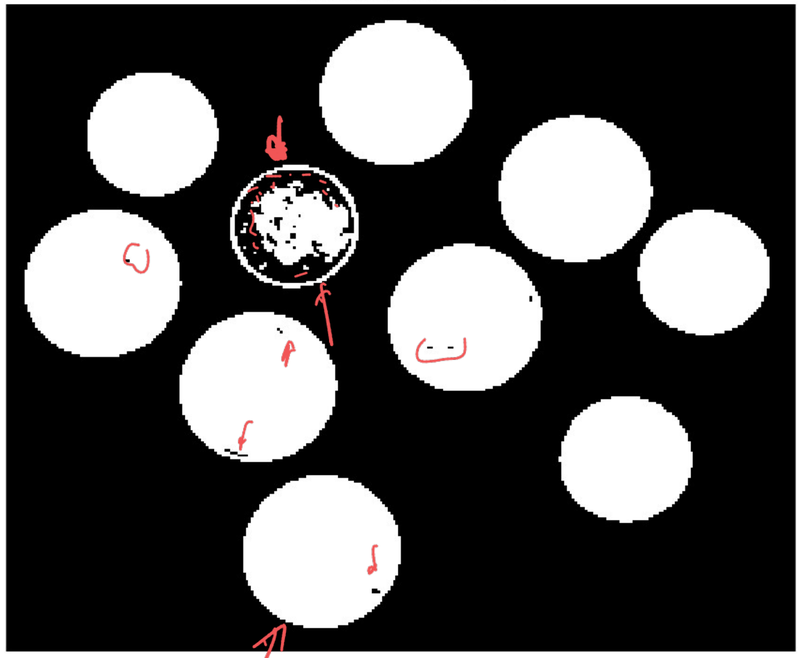
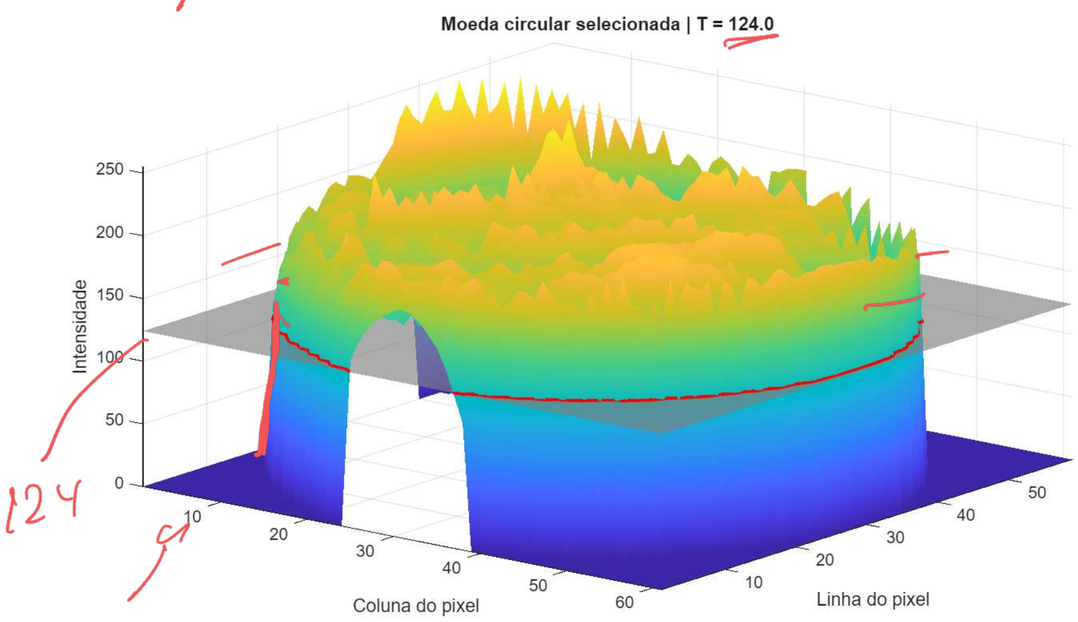


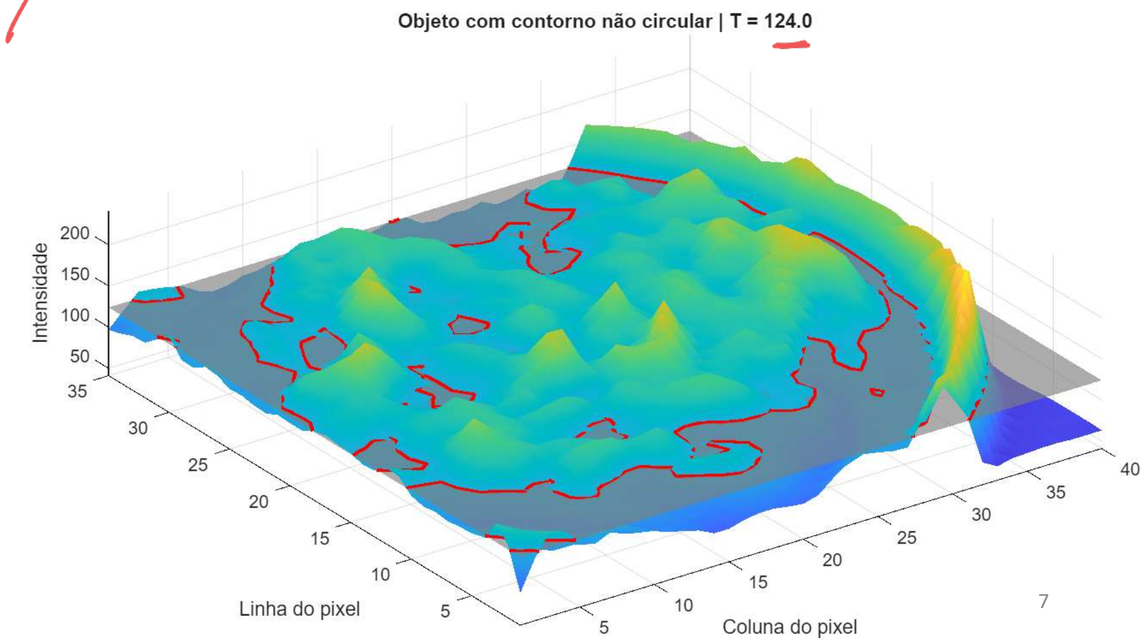
Imagem adaptada de: Geoff Dougherty, Digital Image Processing for Medical Applications, Cambridge University Press, 2009, Figure 9.11.
https://www.cambridge.org/br/academic/subjects/engineering/biomedical-engineering/digital-image-processing-medical-applications?format=HB



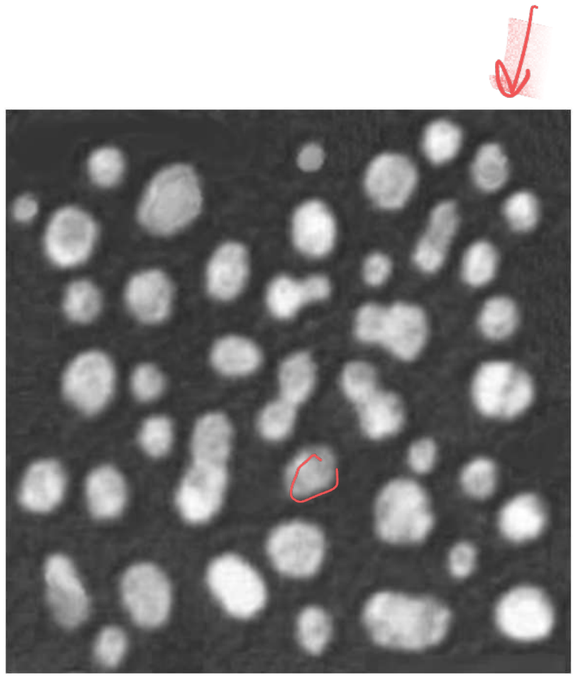



```
BW_no_filter = imbinarize(I, level);
figure;
imshow(BW_no_filter);
title('Threshold sem filtro');
```





Como encontrar o limiar ?



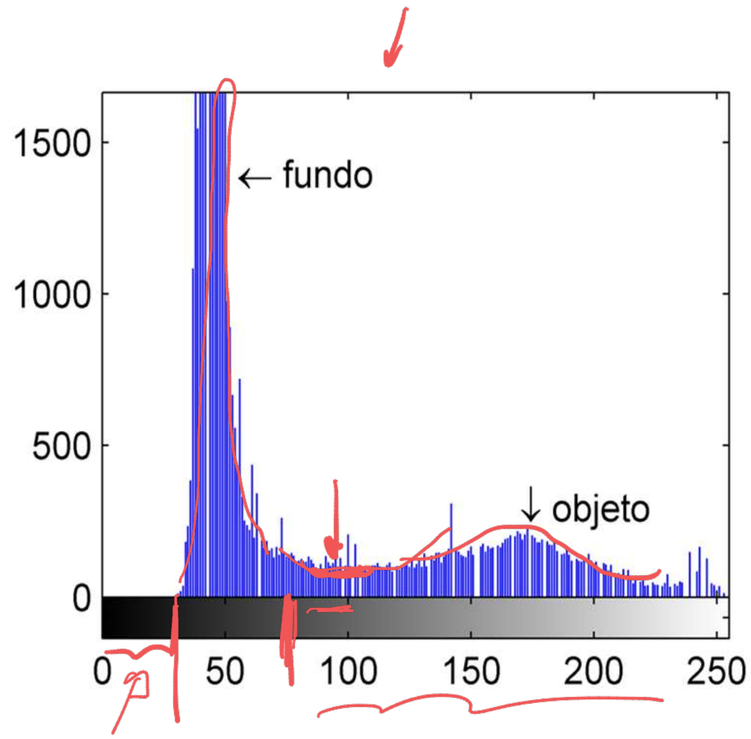


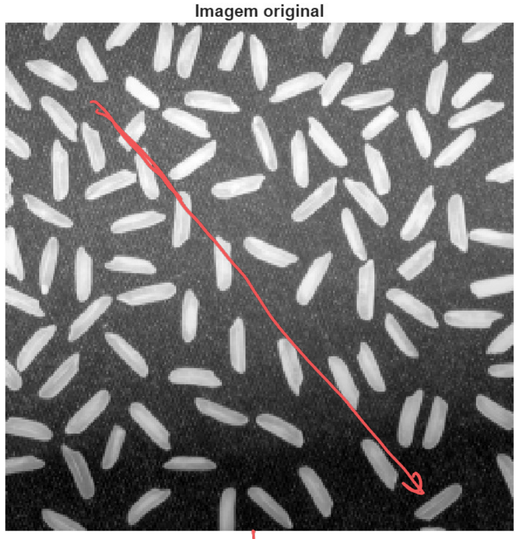
Imagem adaptada de: Geoff Dougherty, Digital Image Processing for Medical Applications, Cambridge University Press, 2009, Figure 9.11.
https://www.cambridge.org/br/academic/subjects/engineering/biomedical-engineering/digital-image-processing-medical-applications?format=HB

Image segmentation: **Limiariação global (thresholding)**

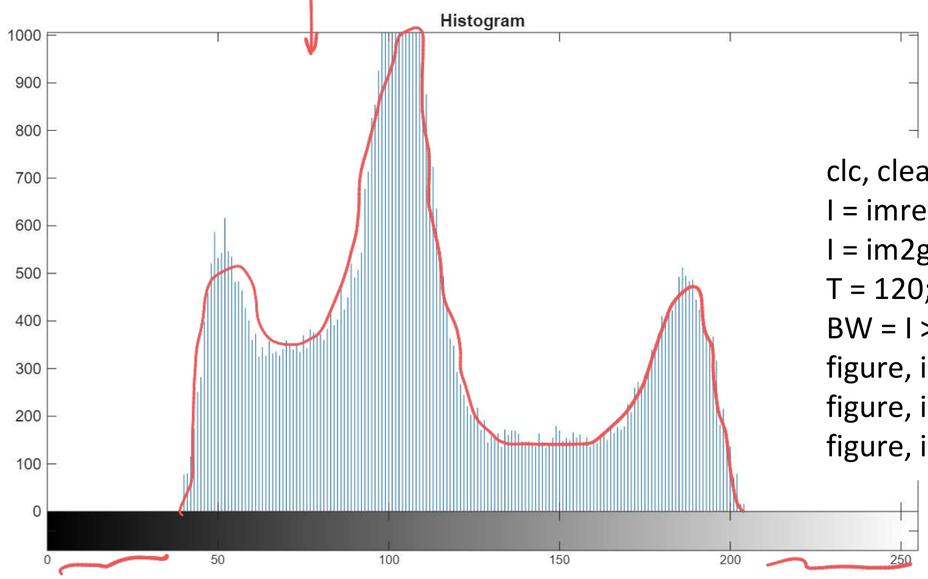
Quando a limiarização simples falha ?

- **Iluminação não uniforme**
- Baixo contraste
- **Histograma não bimodal**

Imagem original

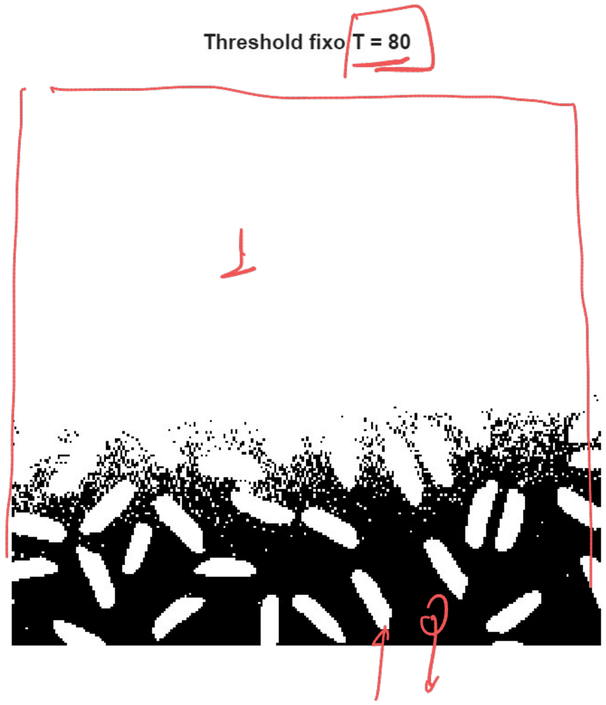


"rice.png"

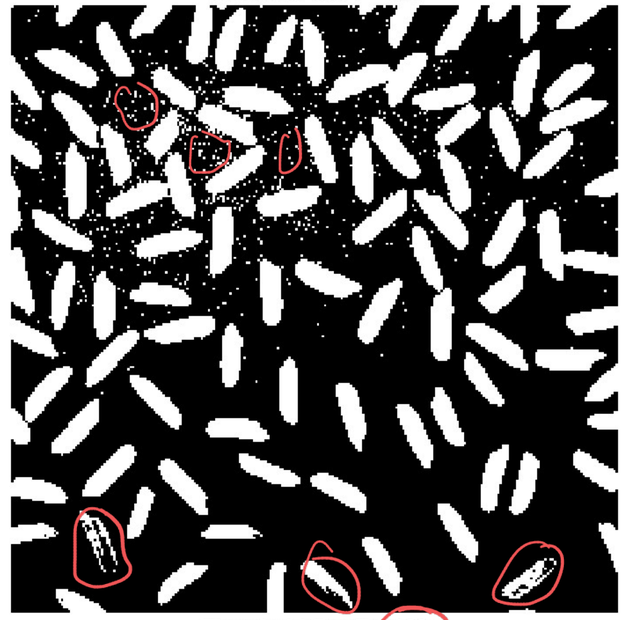



```
clc, clear, close all
I = imread('rice.png');
I = im2gray(I);
T = 120;
BW = I > T;
figure, imshow(I); title('Original');
figure, imshow(BW); title(['Threshold fixo T = ', num2str(T)]);
figure, imhist(I); title('Histograma');
```

Threshold fixo T = 80



Threshold fixo T = 120



Threshold fixo T = 160

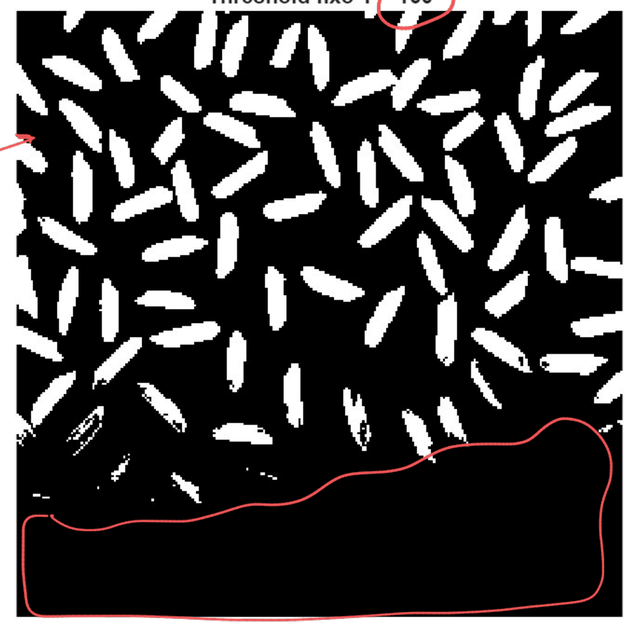
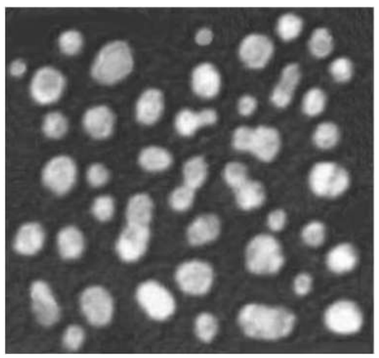


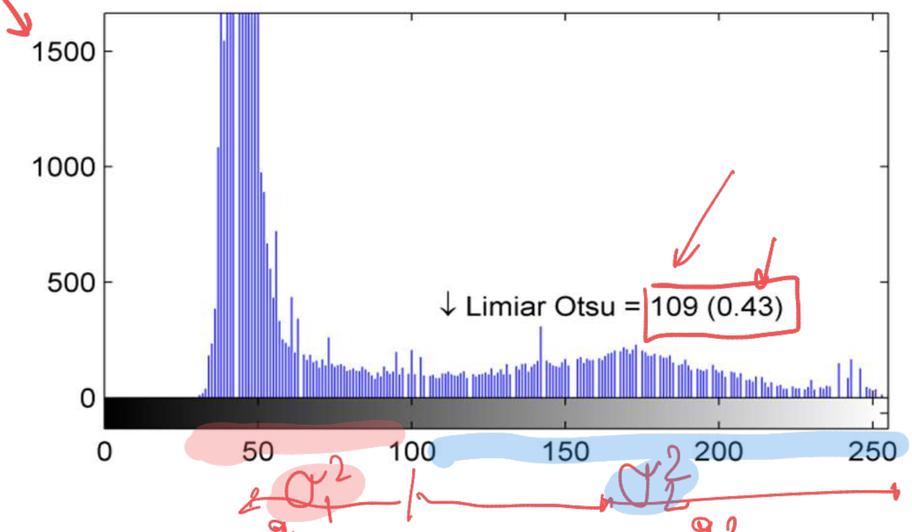
Image segmentation: **Limiariação global (thresholding)**

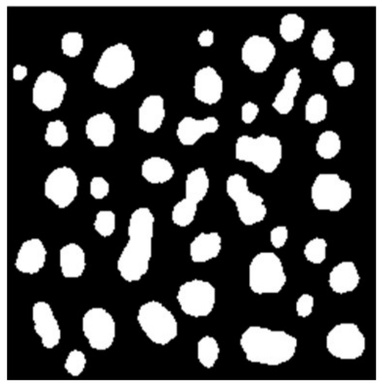
Método de Otsu

Limiarização automática (Otsu)

- Divide pixels em dois grupos
- Minimiza variância dentro dos grupos
- Não precisa escolhere T manualmente







Otsu obtém o nível de cinza que melhor separa, segundo o critério estatístico da variância, os dois grupos de pixels.

O valor ótimo de limiar é aquele que minimiza a variância dentro de cada grupo pixels (pixels mais pareceidos) σ_w^2 :

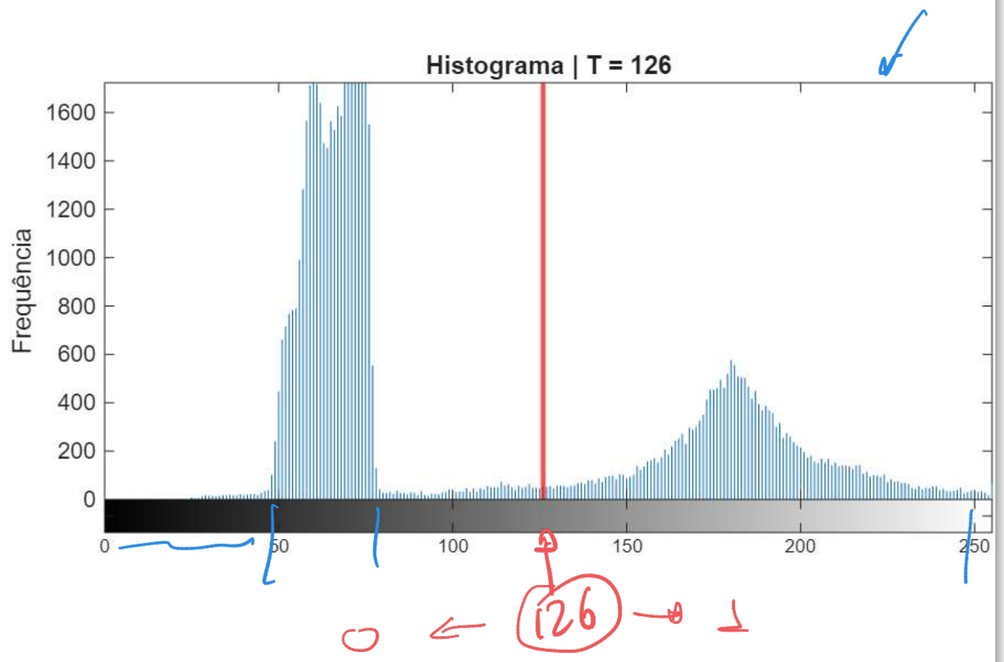
$$\sigma_w^2(t) = q_1(t)\sigma_1^2(t) + q_2(t)\sigma_2^2(t)$$

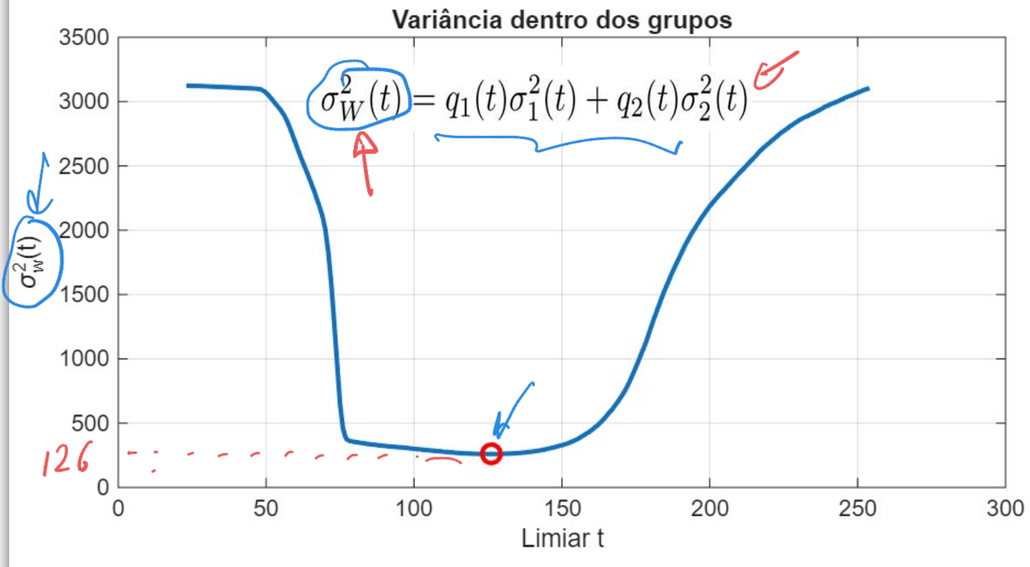
$$q_1 = \sum_{i=0}^{t} p(i) \quad q_2 = \sum_{i=t+1}^{L-1} p(i) \qquad q_1 + q_2 = 1$$

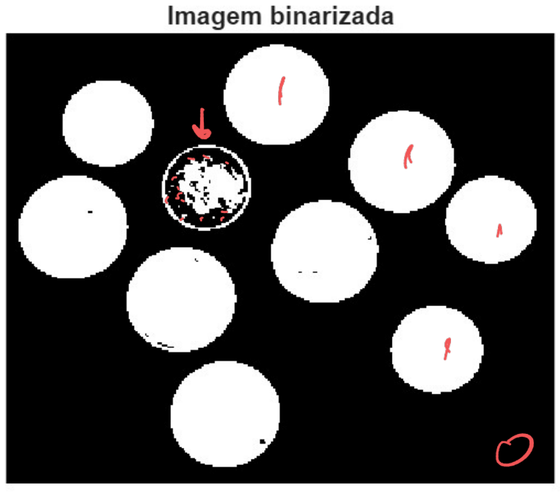
Imagem adaptada de: Geoff Dougherty, Digital Image Processing for Medical Applications, Cambridge University Press, 2009, Figure 9.11.
https://www.cambridge.org/br/academic/subjects/engineering/biomedical-engineering/digital-image-processing-medical-applications?format=HB

Image segmentation: **Limiariação global (thresholding)**

OTSU

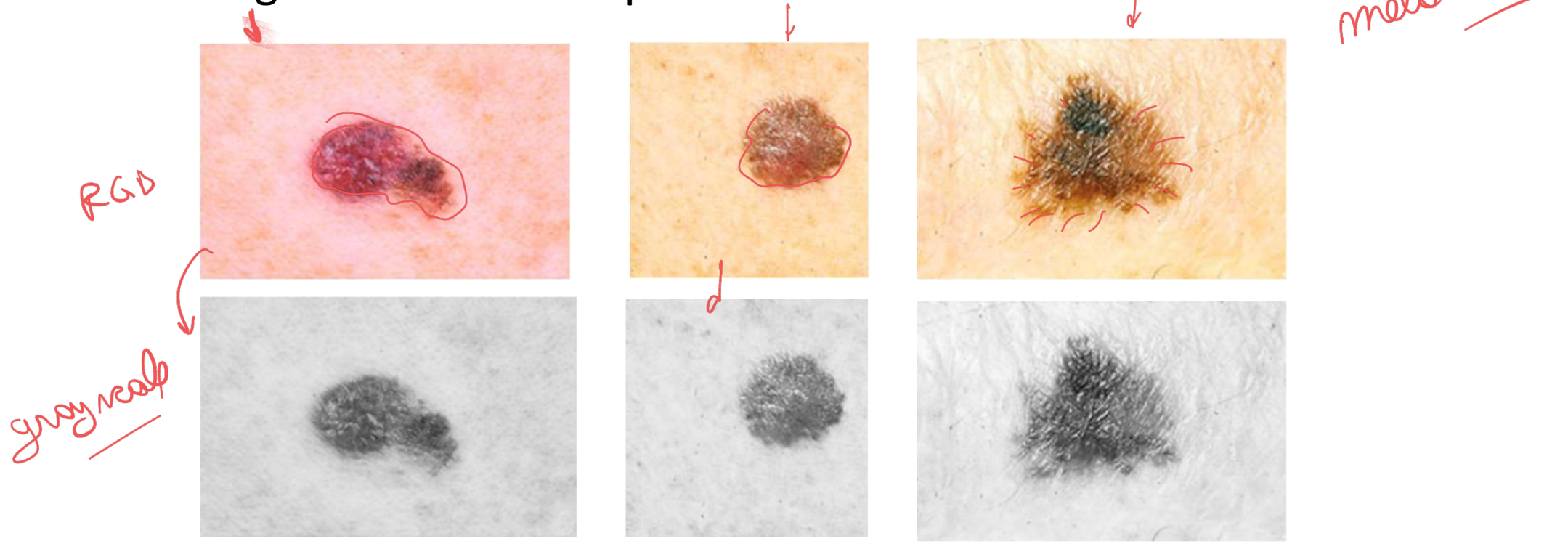






Limiarização global

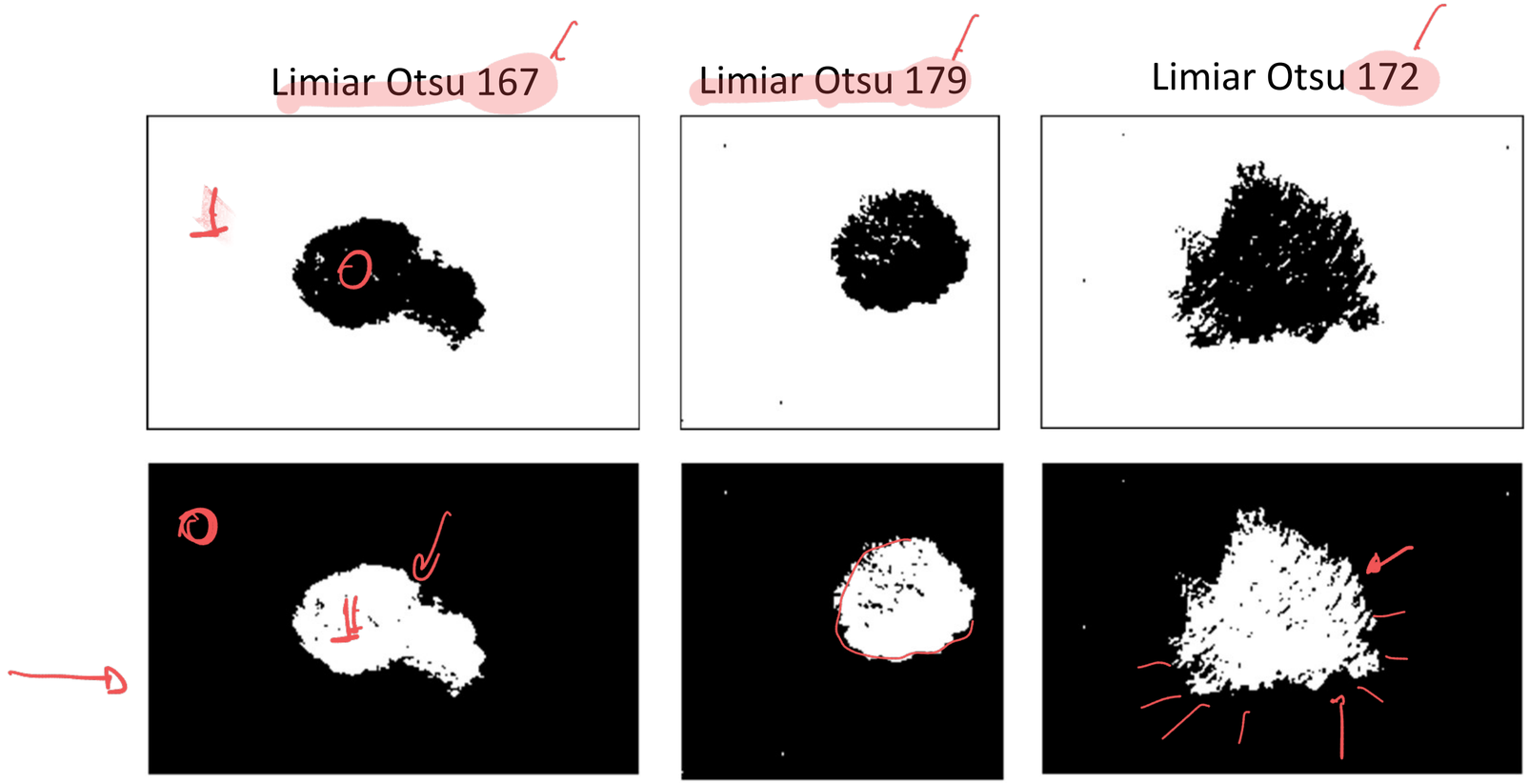
- Usar Otsu para [tentar] segmentar automaticamente as imagens de lesões de pele.



Imagens adaptadas de: Pablo G. Cavalcanti and Jacob Scharcanski, Macroscopic Pigmented Skin Lesion Segmentation and its Influence on the Lesion Classification and Diagnosis, In: Color Medical Image Analysis, Lecture Notes in Computational Vision and Biomechanics Volume 6, 2013, pp 15-39, Edited by M. Emre Celebi and Gerald Schaefer, Springer Netherlands.
https://www.researchgate.net/publication/260713485_Macroscopic_pigmented_skin_lesion_segmentation_and_its_influence_on_lesion_classification_and_diagnosis

Limiarização global

- Usar Otsu para [tentar] segmentar automaticamente as imagens de lesões de pele.



Imagens adaptadas de: Pablo G. Cavalcanti and Jacob Scharcanski, Macroscopic Pigmented Skin Lesion Segmentation and its Influence on the Lesion Classification and Diagnosis, In: Color Medical Image Analysis, Lecture Notes in Computational Vision and Biomechanics Volume 6, 2013, pp 15-39, Edited by M. Emre Celebi and Gerald Schaefer, Springer Netherlands.
https://www.researchgate.net/publication/260713485_Macroscopic_pigmented_skin_lesion_segmentation_and_its_influence_on_lesion_classification_and_diagnosis

Image segmentation: Limiarização Adaptativa (thresholding)

Threshold adaptativo (LOCAL)

Em vez de usar um único limiar para toda a imagem, calcula-se um limiar local para cada região.

$$BW(x,y) = \begin{cases} 1, & I(x,y) > T(x,y) \\ 0, & I(x,y) \leq T(x,y) \end{cases}$$

Quando usar?

- Iluminação não uniforme
- Presença de sombras
- Fundo irregular
- Documentos, microscopia, imagens industriais

Algoritmo de limiarização adaptativa:

1. Definir uma vizinhança local ao redor de cada pixel
2. Calcular uma estatística local da vizinhança
   Ex.: média, mediana ou média ponderada
3. Definir um limiar local T
4. Comparar o pixel com esse limiar
5. Gerar a imagem binária

Matlab:

```
BW = imbinarize(I,'adaptive');

T = adaptthresh(I, 0.5, 'NeighborhoodSize', N);
BW = imbinarize(I,T);
```

Por padrão o MATLAB tralaha com a média local ponderada (gaussian weighted)

O 0.5 é o parâmetro de sensibilidade (controlo o nível do limiar): baixo (0.3) menos pixels=1, alto(0.7) mais pixels=1

$$T(x,y) \approx \mu(x,y) \cdot (1-k)$$

Para uma imagem de $M \times N$:

Nhood = 2 * floor([M N]/16) + 1; (*valor padrão de limiar*)

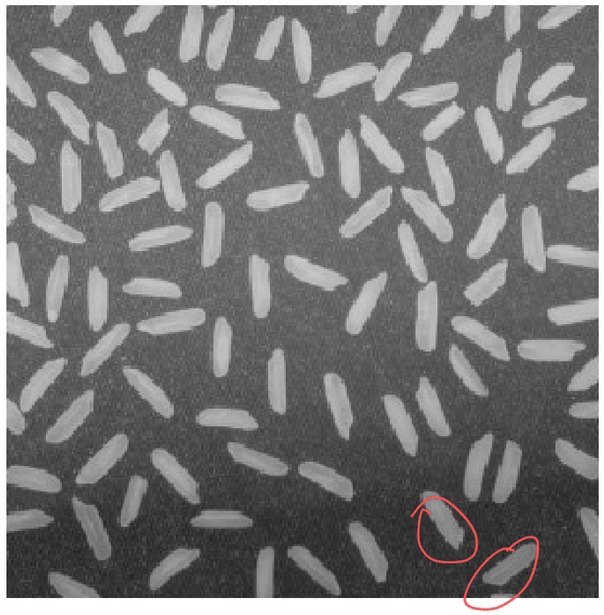
Imagem 256 x 256

NeighborhoodSize = [33 33]

Image segmentation: L**imiarização Adaptativa (thresholding)**

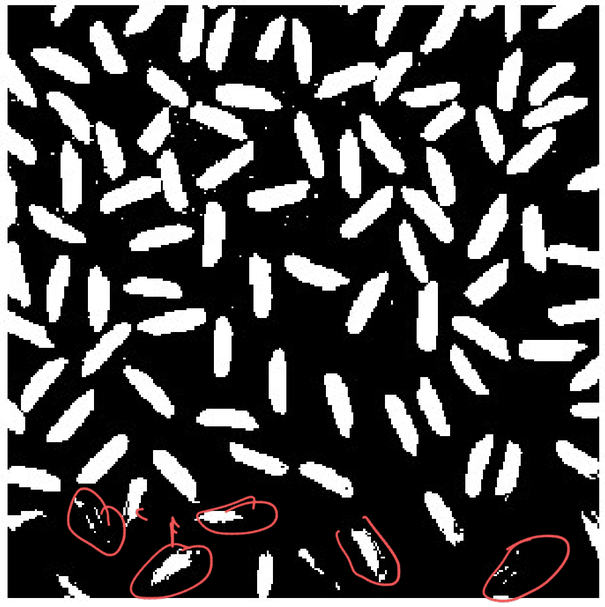
Comparação: Otsu vs Adaptativo

Imagem original



I = imread('rice.png');
imshow(I)

Otsu: limiar global



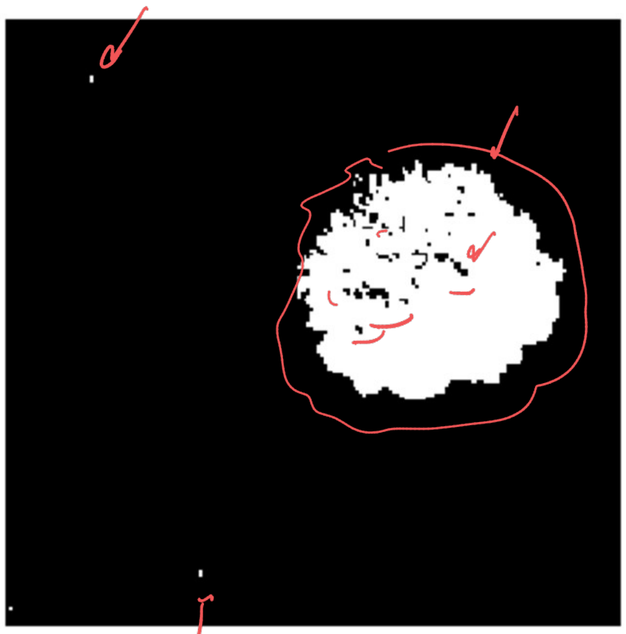
level = graythresh(I);
BW_otsu = imbinarize(I, level);
imshow(BW_otsu)

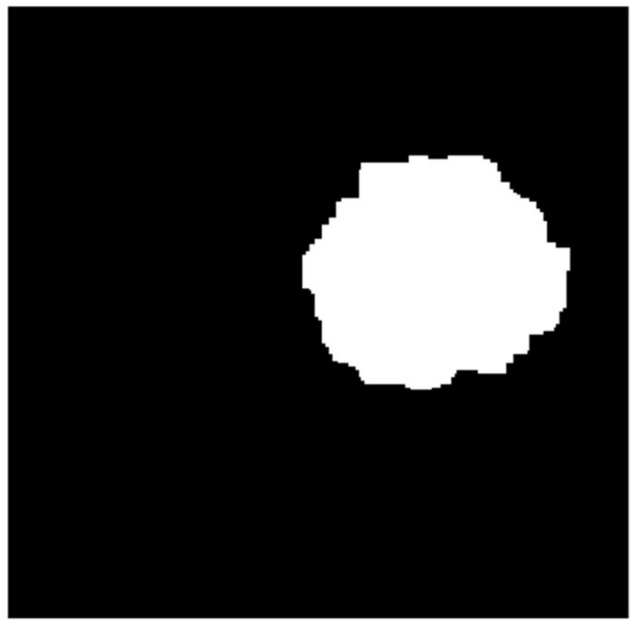
Adaptativo: limiar local

T = adaptthresh(I, 0.45);
BW_adapt = imbinarize(I, T);
imshow(BW_adapt)

Operações morfológicas

- Morfologia significa "estudo da forma, estrutura, aparência". Então, as operações morfológicas tem o objetivo de analisar e manipular a forma (*shape*) dos objetos na imagem.





Imagens adaptadas de: Pablo G. Cavalcanti and Jacob Scharcanski, Macroscopic Pigmented Skin Lesion Segmentation and its Influence on the Lesion Classification and Diagnosis, In: Color Medical Image Analysis, Lecture Notes in Computational Vision and Biomechanics Volume 6, 2013, pp 15-39, Edited by M. Emre Celebi and Gerald Schaefer, Springer Netherlands.
https://www.researchgate.net/publication/260713485_Macroscopic_pigmented_skin_lesion_segmentation_and_its_influence_on_lesion_classification_and_diagnosis

Operações morfológicas

- A base matemática da morfologia é a teoria de conjuntos.

  Um conjunto de pixels binários denominado **elemento estruturante (*structuring element*)** varre a imagem (outro conjunto de pixels binários) e as operações morfológicas são realizadas entre o elemento estruturante e os pixels da imagem contidos no elemento estruturante.

- Este processo de 'varrer a imagem com o elemento estruturante' é similar àquele que realizamos em um *box filter*, por exemplo. Mas ao invés de aplicar uma convolução, **aplicamos a operação morfológica entre máscara (elemento estruturante) e os pixels da imagem.**

- O **SE** pode ter um **formato arbitrário** e depende da aplicação.

Operações morfológicas

- **Imagens binárias**: Trataremos de **operações morfológicas** aplicadas a **imagens binárias** (BW).
  - Pixels 1 (branco): são os **pixels dos objetos**.
  - Pixels 0 (preto): são os **pixels do fundo**

- **Hot spot**: é o **ponto de referência** do elemento estruturante (SE). Este ponto de referência nem sempre é o ***ponto central* do elemento estruturante**.
- **Elemento estruturante:**
  - um ponto ativo tem valor ‘1’;
  - um ponto não ativo (irrelevante) tem valor ‘0’**.**

Operações morfológicas

Vamos ver 5 operações morfológicas:

- Dilatação (delation)
- Erosão (erosion)
- Opening
- Closing
- Opening-Closing

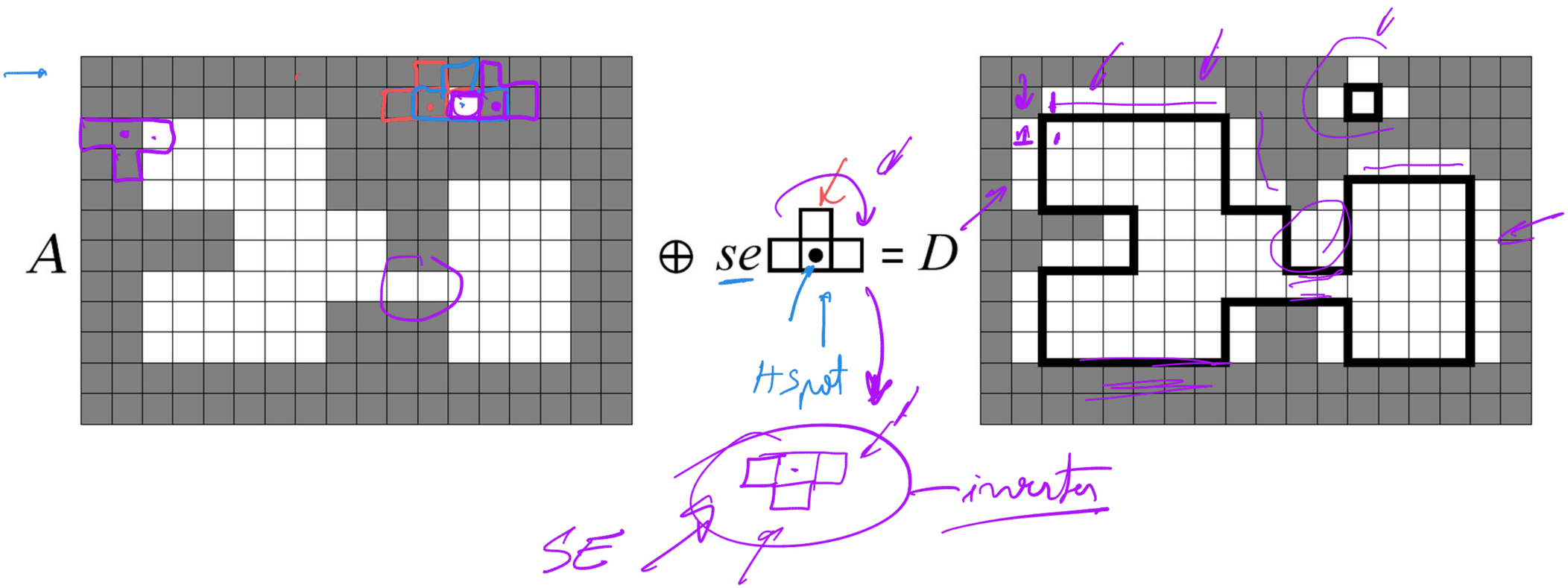
Operações morfológicas: Dilatação (dilation)

Dilatação (Dilation)
- "Engrossa" o objeto
- Expande regiões
- Preenche pequenos buracos

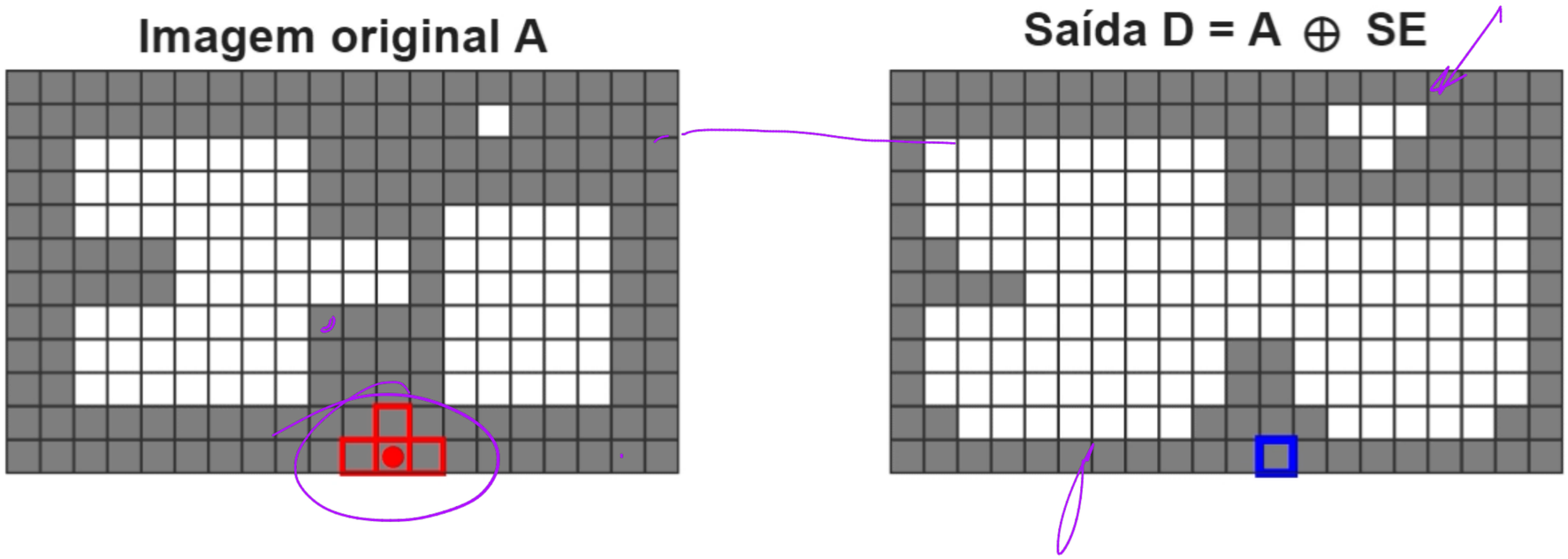
$$D = A \oplus SE$$

Procedimento da dilatação de *A* por um elemento estruturante *SE*:
1. Posicione o **centro do SE** sobre um pixel
2. Observe todos os pixels cobertos pelo SE
3. Se pelo **menos um for 1** → o **pixel central** na saída vira 1

Dilatação funciona como um OR local



Dilatação morfológica: o SE varre a imagem



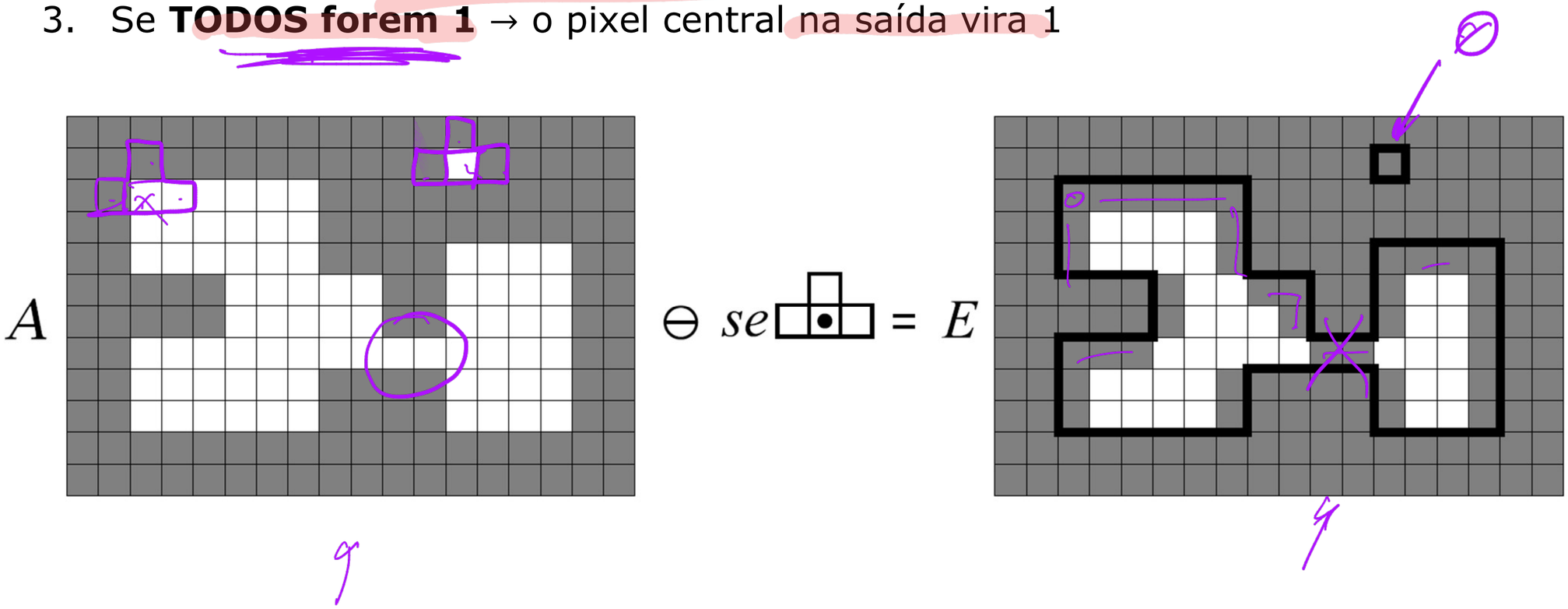
dilatacao_GIF.gif

Operações morfológicas: Erosão (erosion)

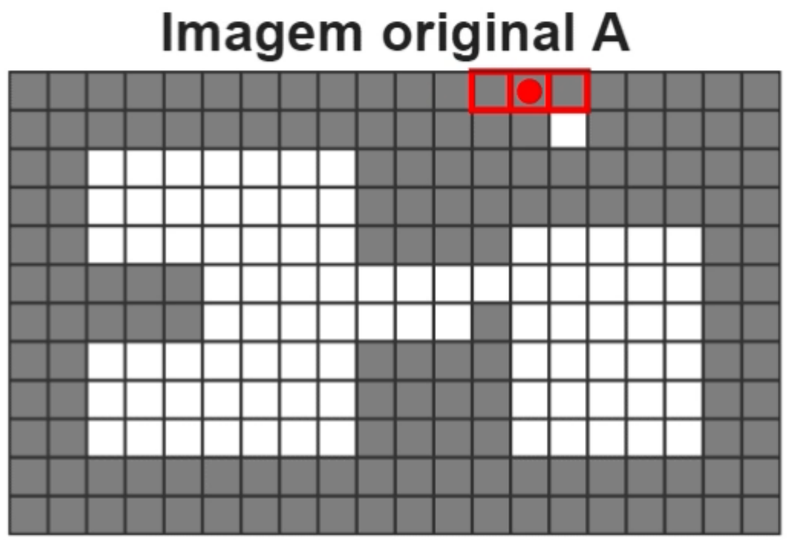
Erosão (Erosion)
- ✓ Encolhe os objetos
- ✓ Remove ruídos pequenos

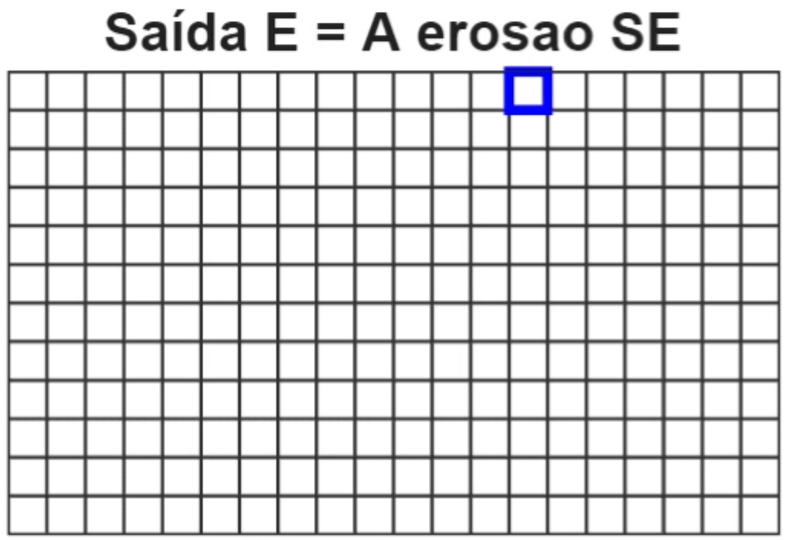
$$E = A \ominus se$$

Procedimento da Erosão de A por um elemento estruturante SE:
1. Posicione o **centro do SE** sobre um pixel
2. Observe todos os pixels cobertos pelo SE
3. Se **TODOS forem 1** → o pixel central na saída vira 1



Erosão morfológica





erosao_GIF_objetos_conectados.gif

Operações morfológicas: **Abertura (Opening)** → **Limpa a imagem sem crescer os objetos**

Definição: Erosão seguida de dilatação

Forma matemática: $A \circ SE = (A \ominus SE) \oplus SE$

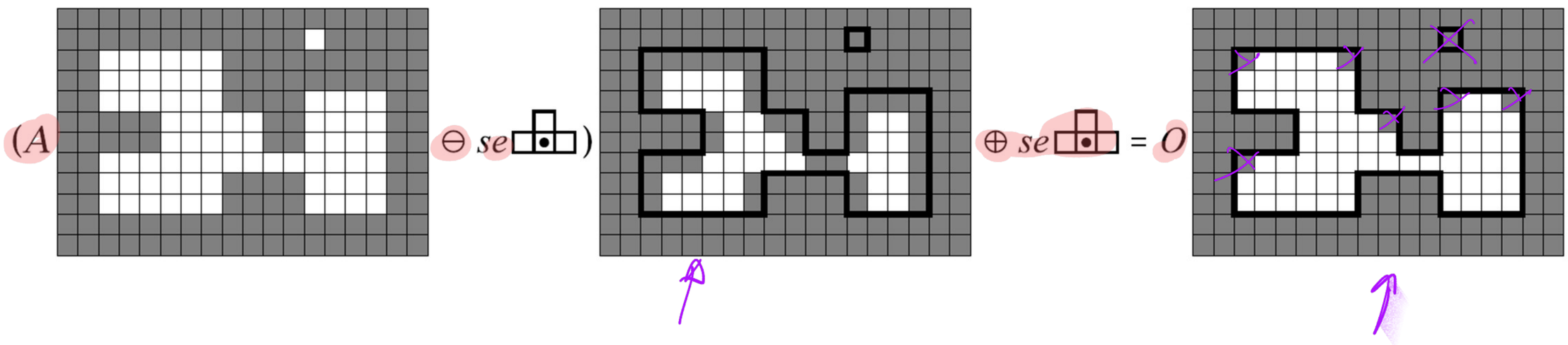
Efeitos:

- Remove **estruturas pequenas**
- Elimina **ruído fino**
- Quebra **conexões finas**
- Suaviza contornos sem expandir significativamente o objeto

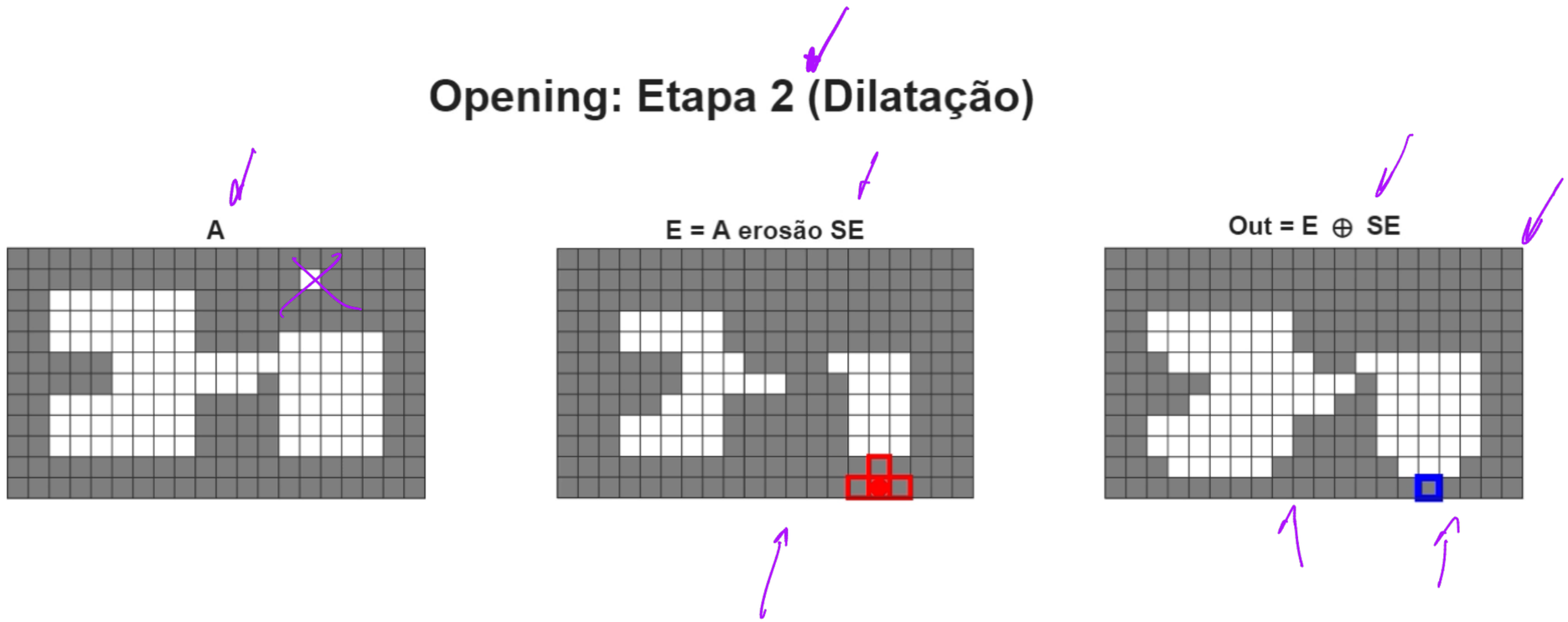
Procedimento:

Etapa 1 – Erosão
Todos os pixels cobertos pelo SE for 1 → saída 1
Caso contrário → 0
imagem encolhida

Etapa 2 — Dilatação
Pelo menos um pixel coberto pelo SE for 1 → saida 1
imagem volta a crescer
(sem recuperar o que foi perdido)



Operações morfológicas: **Abertura (Opening)** → **Limpa a imagem sem crescer os objetos**



opening_GIF_SE.gif

Operações morfológicas: **Fechamento (Closing)** **Preenche buracos sem reduzir os objetos**

Definição: Dilatação seguida de erosão

Forma matemática: $A \cdot SE = (A \oplus SE) \ominus SE$

Efeitos:

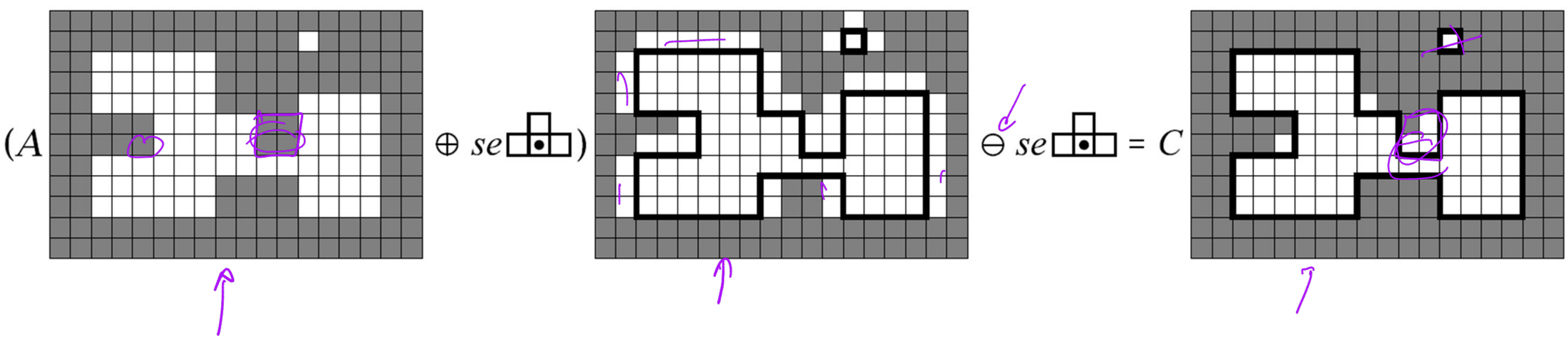
- Preenhe buracos
- Conecta regiões próximas
- Suaviza contornos internos

Pequenos buracos são preenchidos e conexões são reforçadas

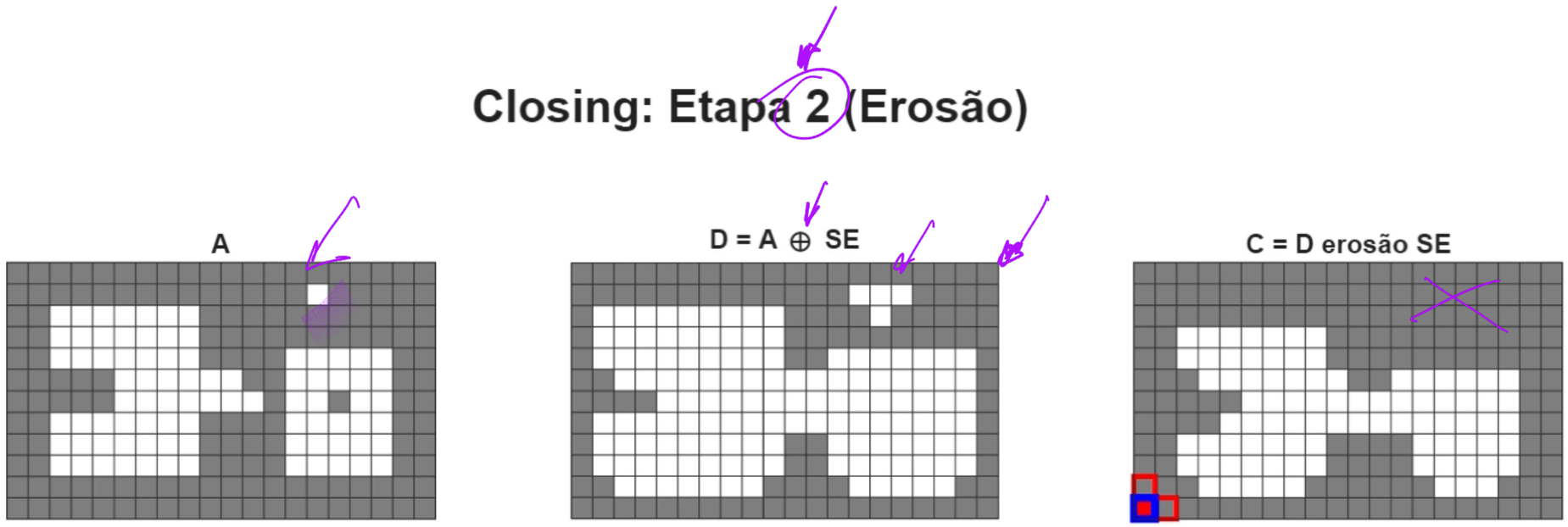
Procedimento:

Etapa 1 — Dilatação
Pelo menos um pixel coberto pelo SE for 1→ saída 1
imagem expande

Etapa 2 – Erosão
Todos os pixels cobertos pelo SE forem 1 → saída 1
Caso contrário → 0
imagem encolhe



Operações morfológicas: **Fechamento (closing)** **Preenche buracos sem reduzir os objetos**

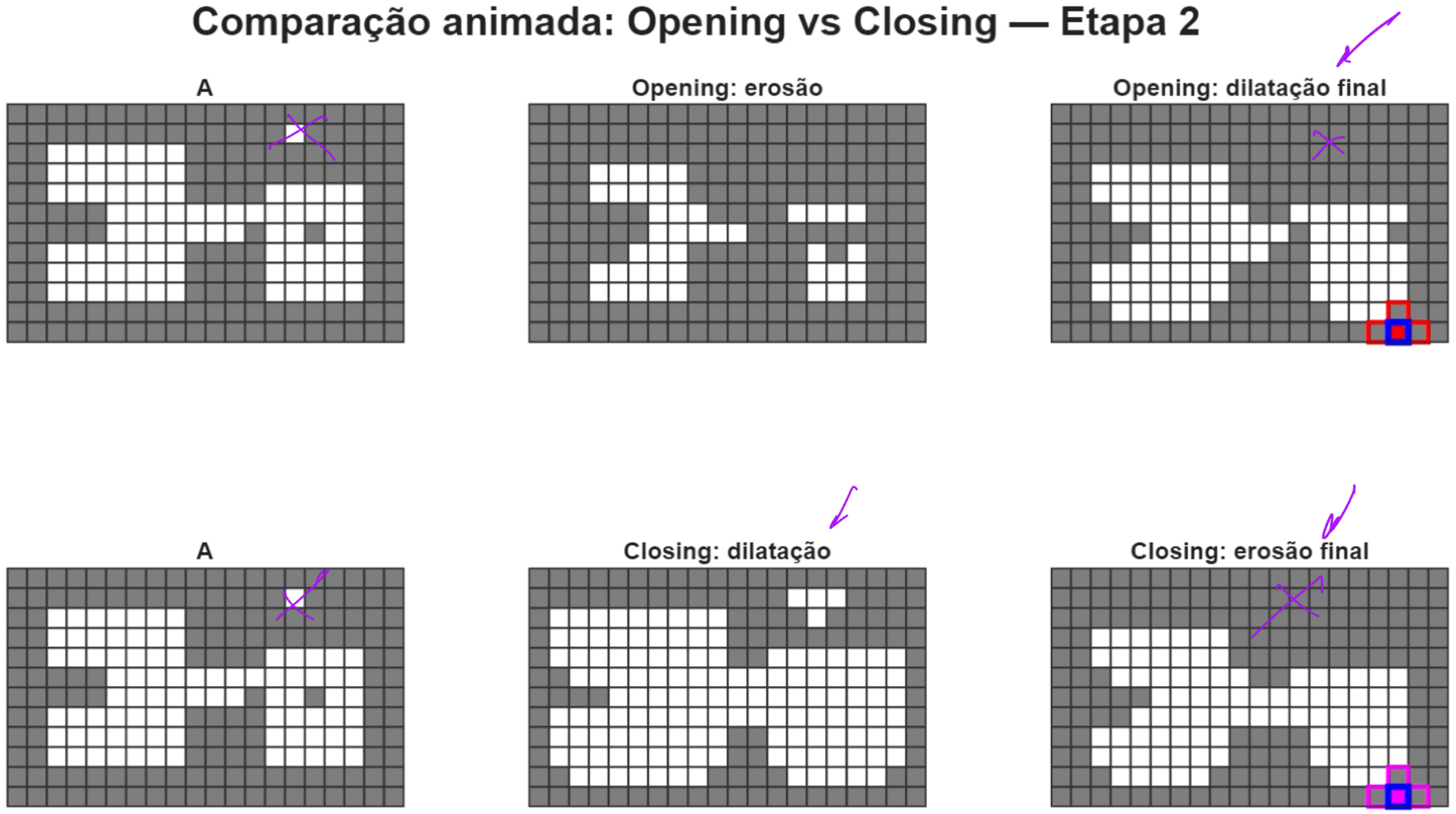


closing_GIF.gif

Operações morfológicas: **Comparação das operações vistas**

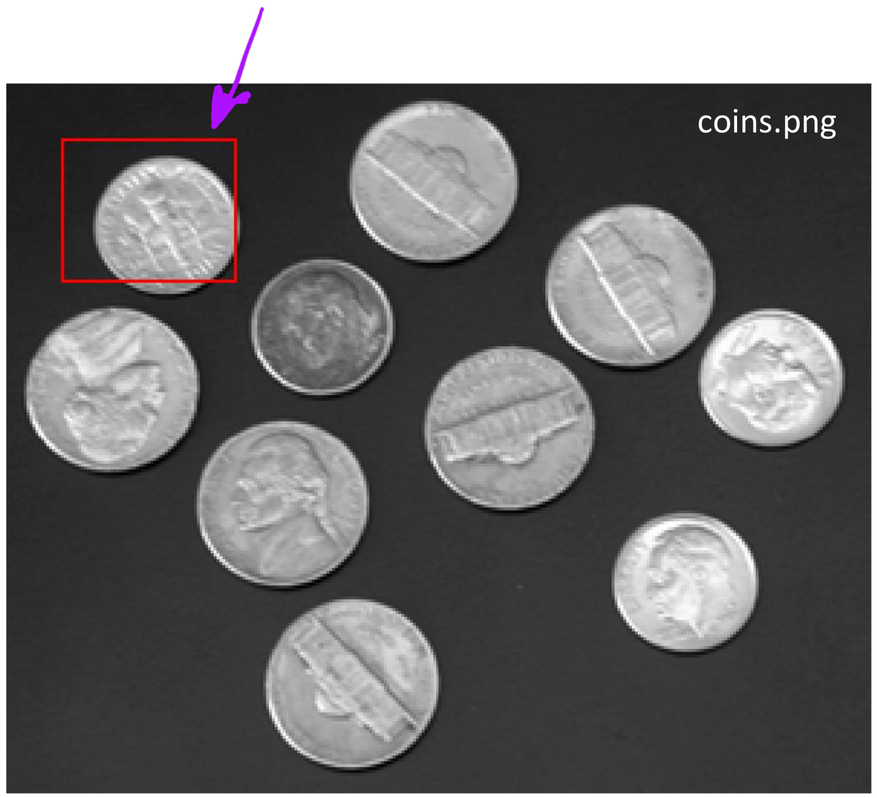
Operação	Efeito
Dilatação	cresce
Erosão	reduz
Abertura	limpa
Fechamento	preenche

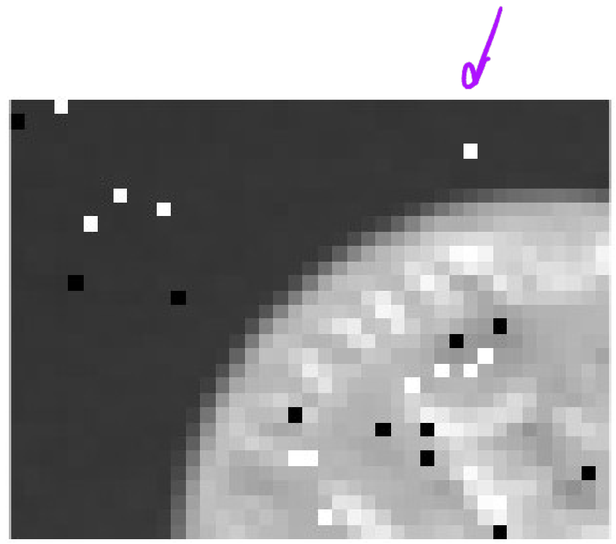
Operações morfológicas: Opening vs Closing



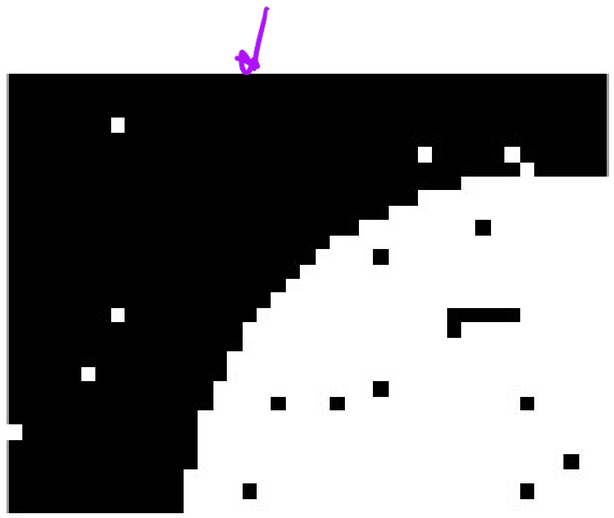
opening_vs_closing_animado.gif

Operações morfológicas: Erosão (erosion) – Exemplo prático





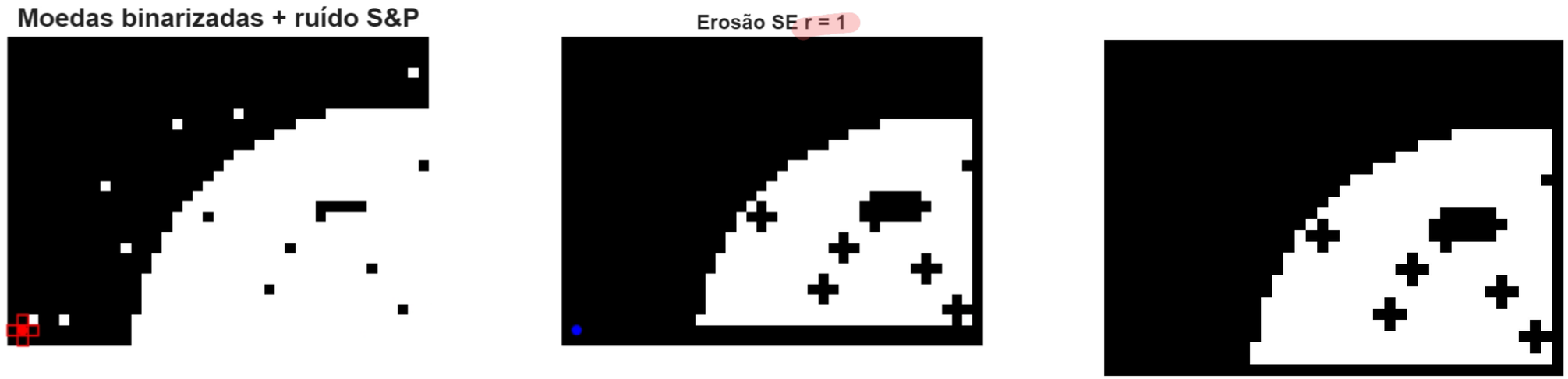
I_noisy = imnoise(Igray, 'salt & pepper', 0.02);



BW = imbinarize(I_noisy, 0.6);

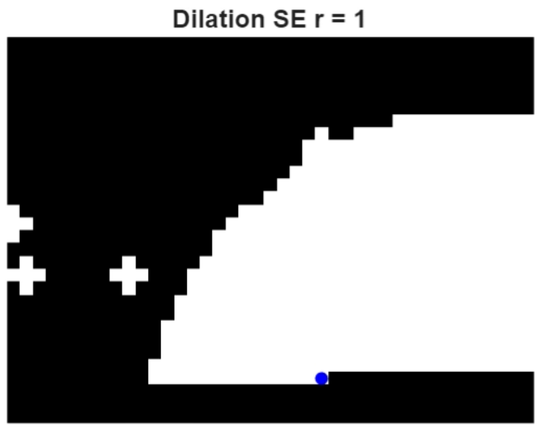
```
Igray = Igray(20:floor(M/5), 20:floor(N/5));
rectangle('Position',[20 20 floor(N/5) floor(M/5)], 'EdgeColor','r', 'LineWidth',2);
```

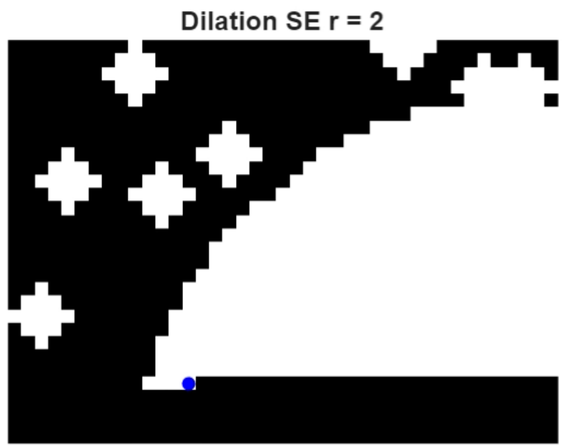
Operações morfológicas: Erosão (erosion) – Exemplo prático



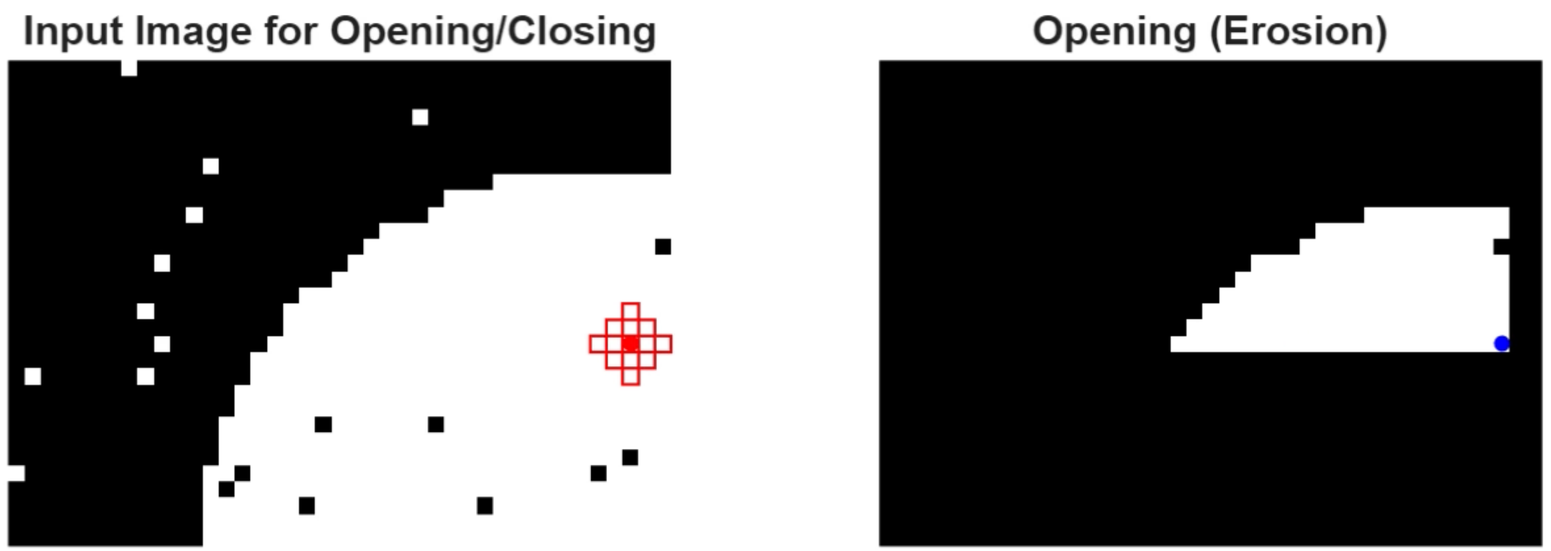
erosao_moedas_SE_circular_r1_used.gif erosao_moedas_SE_circular_r2_used.gif

Operações morfológicas: Dilatação (Dilation) – Exemplo prático



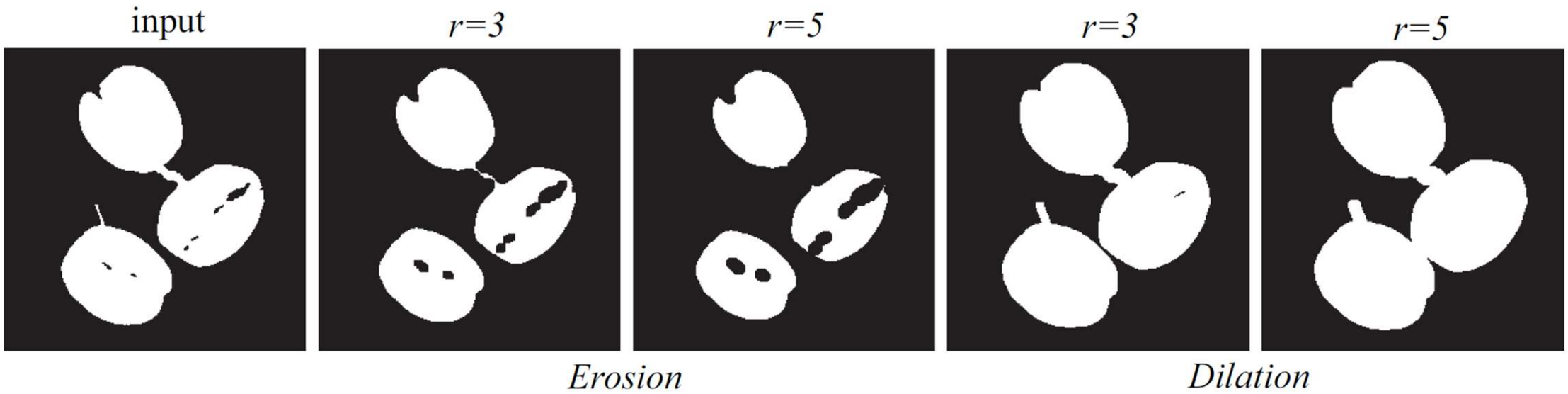


Operações morfológicas: Opening-Closing



Operações morfológicas: **Erosão e Dilatação com diferentes tamanhos de SE**

O tamanho do SE controla a intensidade da operação



Operações morfológicas

Original

Limiarizada (binária)

Dilatação

Erosão

Fechamento

Fechamento: funde quebras estreitas, elimina pequenos buracos, preenche espaços de contornos.

Abertura

Abertura: quebra ligações estreitas, elimina protrusões finas.

Imagens: https://www.cs.auckland.ac.nz/courses/compsci773s1c/lectures/ImageProcessing-html/topic4.htm

Operações morfológicas: Open-close filter

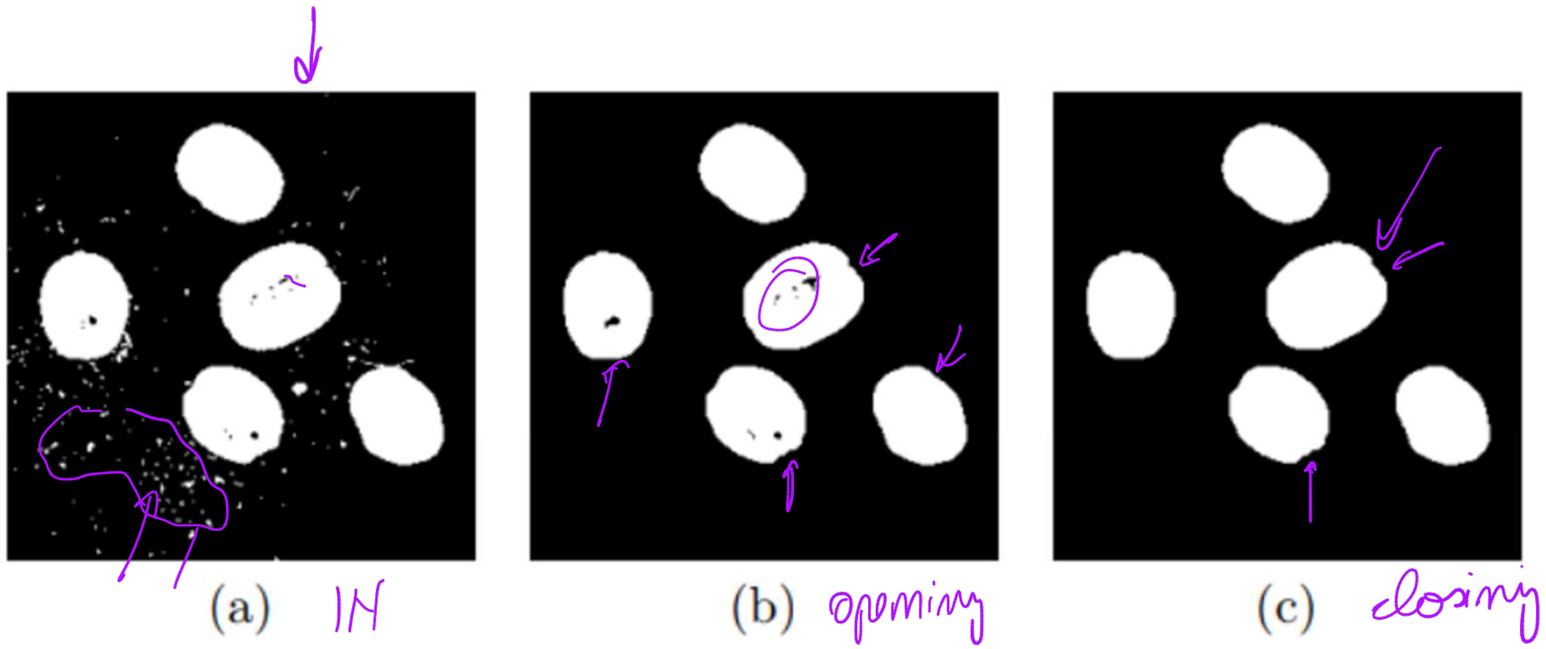


Figure 24.5: Open-close filtering using a SE of disk shape with radius $r = 5$. (a) Input image; (b) opening; (c) subsequent closing.

Esqueletização (Skeleton)

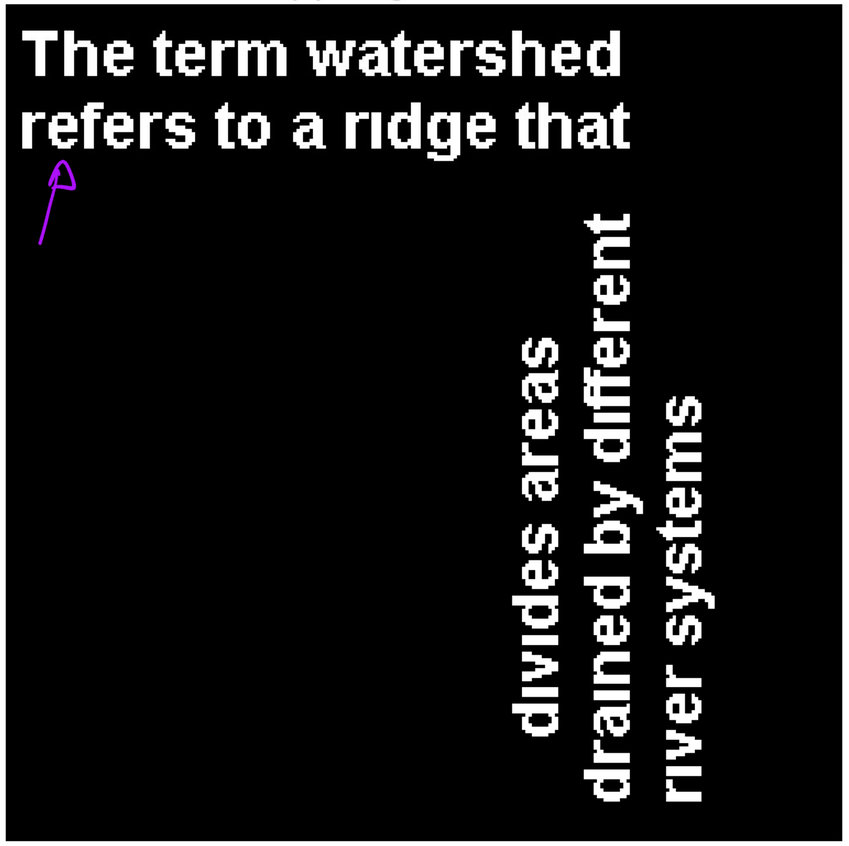
A esqueletização é uma operação morfológica que busca reduzir objetos binários a estruturas finas, representativas de sua geometria.

Obs.: Dependendo do algoritmo utilizado, a preservação da conectividade e a espessura do esqueleto podem variar.

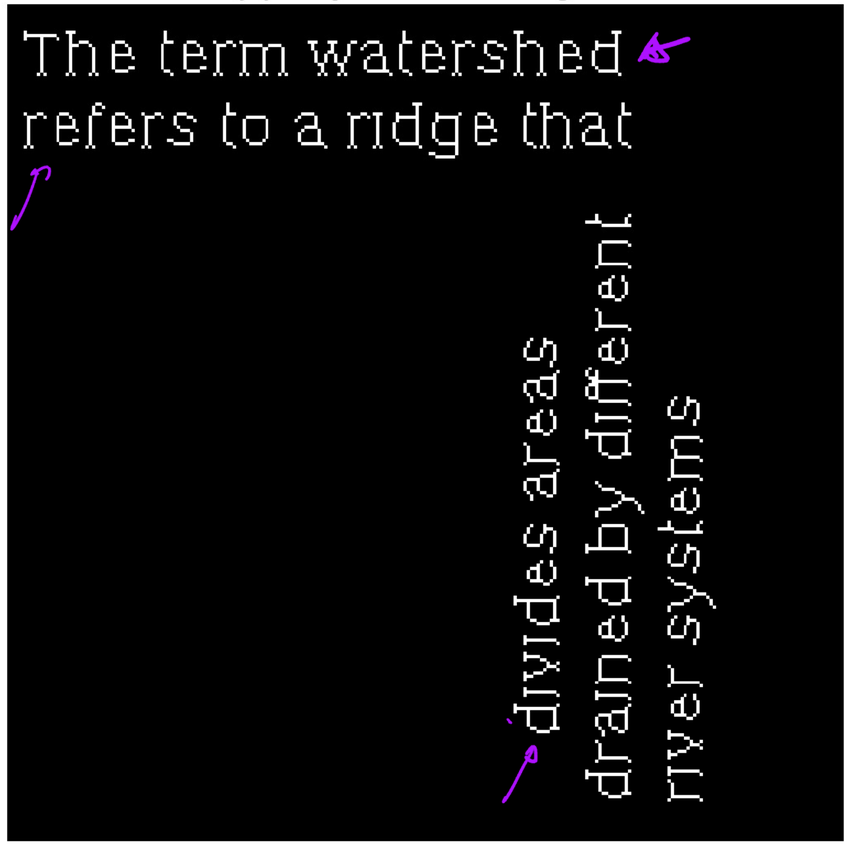
O esqueleto procura preservar:

- Conectividade.
- Estrutura de ramificações.
- Organização geométrica do objeto.

(a) Imagem binária



(b) Esqueleto morfológico



Esqueletização (Skeleton)

$$\left.\begin{aligned} A_1 &= A \ominus B \\ A_2 &= A_1 \ominus B \\ A_3 &= A_2 \ominus B \\ A_4 &= A_3 \ominus B \end{aligned}\right\}$$

A cada iteração, o objeto diminui.

Erosões sucessivas podem romper conexões e eliminar completamente alguns componentes.

A erosão reduz os objetos de acordo com o elemento estruturante.

A esqueletização busca produzir uma representação fina da estrutura, preservando propriedades topológicas.

Esqueletização (Skeleton): esqueletizaqção utilizando operações morfológicas

Erosão após k iterações:

$$A_k = A \ominus k\,B$$

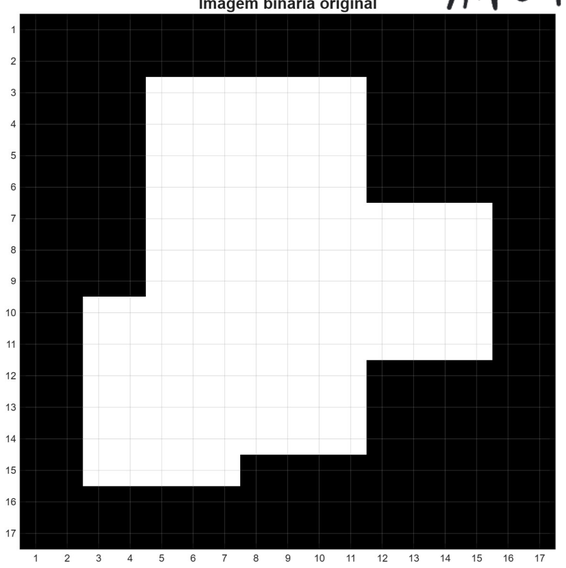
1. Obtemos A_k após k erosões, sendo $A_0 = A$
2. Aplicamos uma abertura sobre A_k
3. Subtraímos a abertura de A_k
4. Guardamos os pixels restantes
5. Repetimos enquanto a erosão não eliminar completamente oobjeto

$S_k(A) = A_k \setminus (A_k \circ B))$, onde \ representa a diferença entre conjuntos → MATLAB: Sk = Ak & ~imopen(Ak,B);

$$S(A) = \bigcup_{k=0}^{K} S_k(A)$$

Lê-se: O esqueleto de A é igual à união dos conjuntos S índice k de A, para k variando de zero até K.
MATLAB: S = S | Sk;

$S(A)$: esqueleto final da imagem binária A
∪: operador de união de conjuntos
$k = 0$: primeira iteração
K: última operação
$S_k(A)$: conjunto de pixels pertencentes ao esqueleto obtido na iteração k

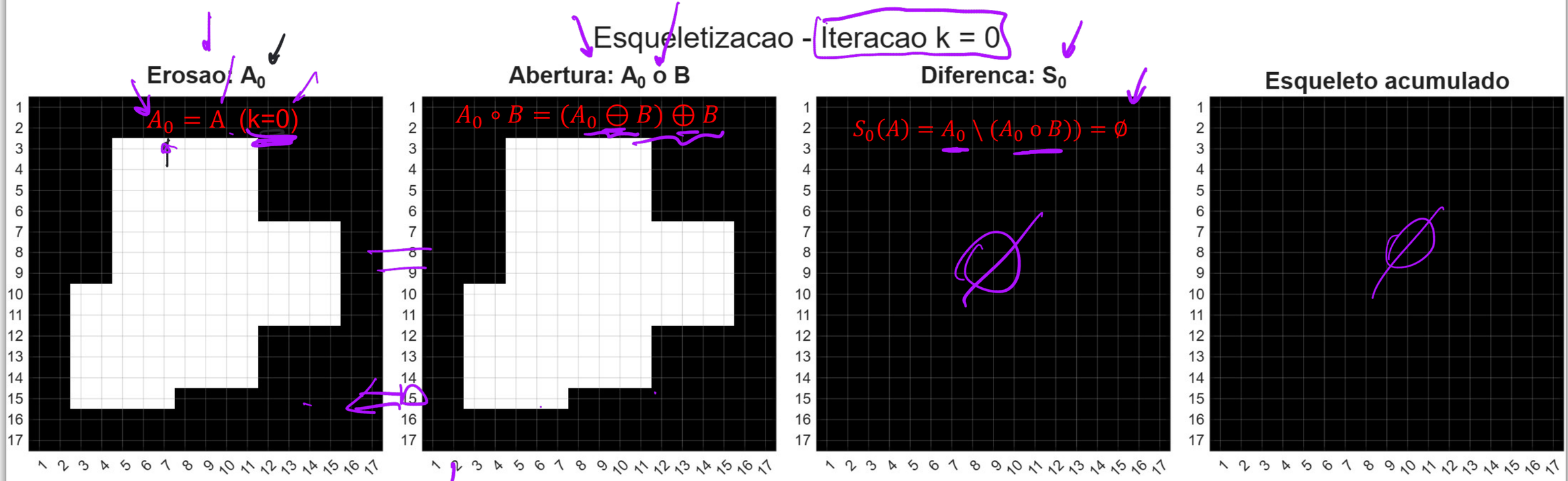


INPUT = A

SE

$$B = \begin{matrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{matrix}$$

Esqueletizacao - Iteracao k = 0

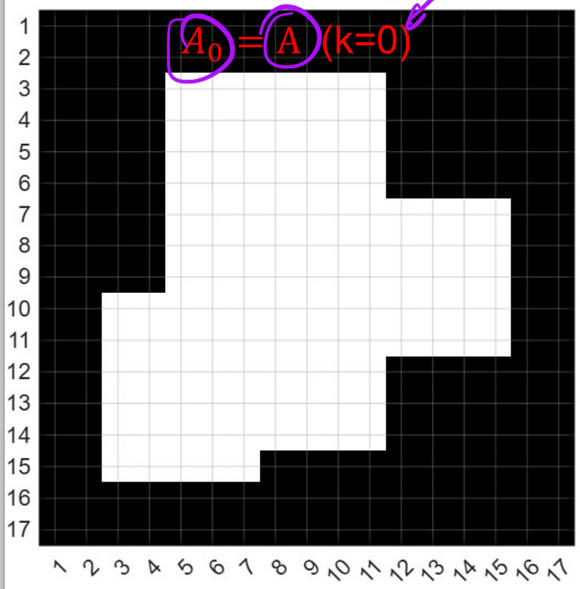


$B =$

Esqueletizacao - Iteracao k = 0

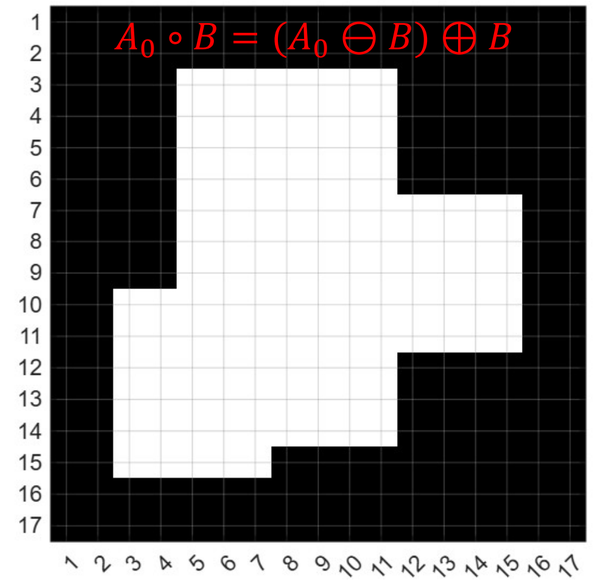
Erosao: A_0

$A_0 = A$ (k=0)



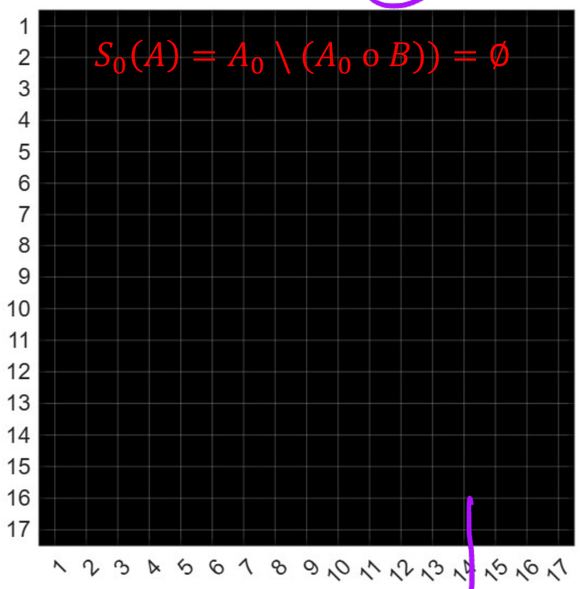
Abertura: A_0 o B

$A_0 \circ B = (A_0 \ominus B) \oplus B$

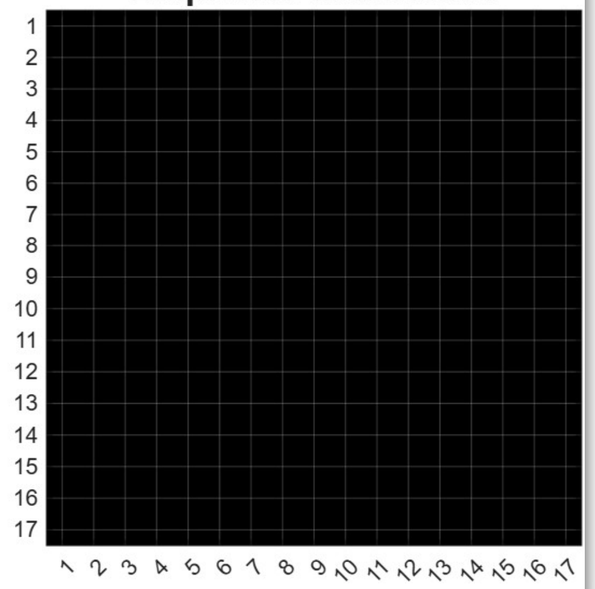


Diferenca: S_0

$S_0(A) = A_0 \setminus (A_0 \circ B)) = \emptyset$



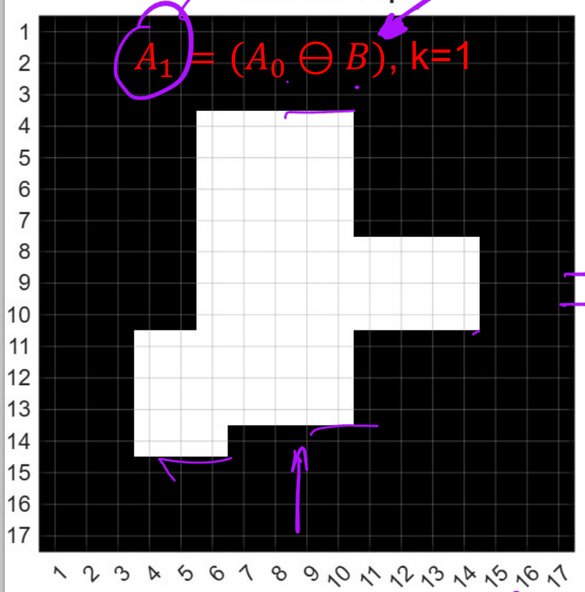
Esqueleto acumulado



Esqueletizacao - Iteracao k = 1

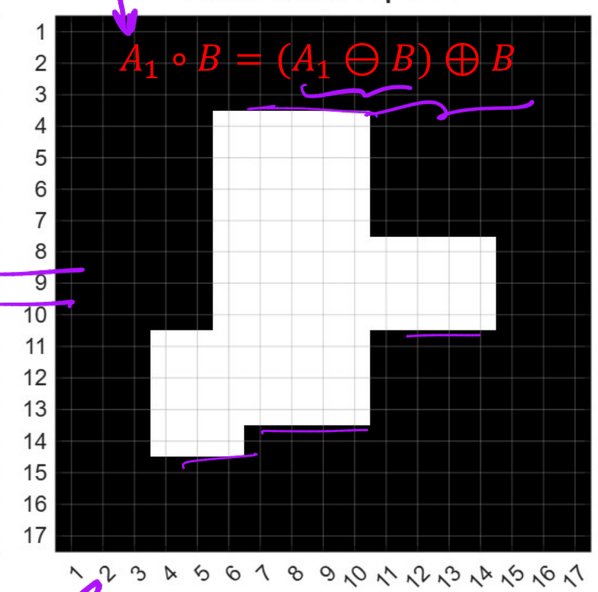
Erosao: A_1

$A_1 = (A_0 \ominus B)$, k=1



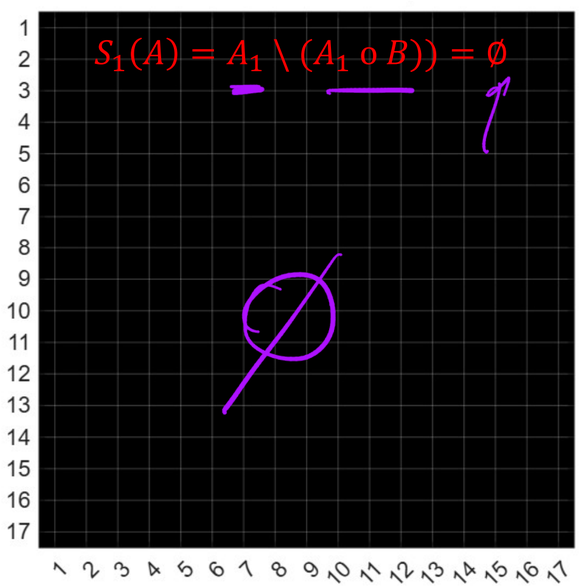
Abertura: A_1 o B

$A_1 \circ B = (A_1 \ominus B) \oplus B$



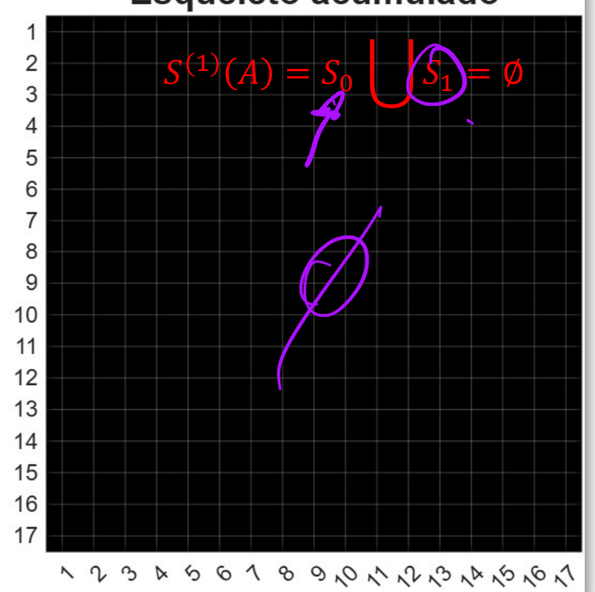
Diferenca: S_1

$S_1(A) = A_1 \setminus (A_1 \circ B)) = \emptyset$



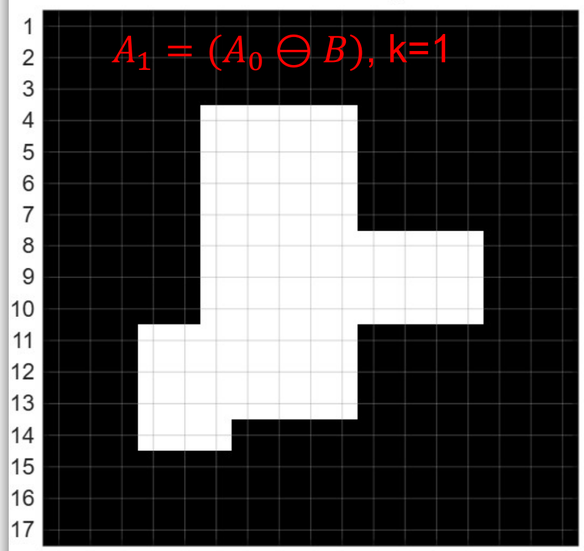
Esqueleto acumulado

$S^{(1)}(A) = S_0 \cup S_1 = \emptyset$

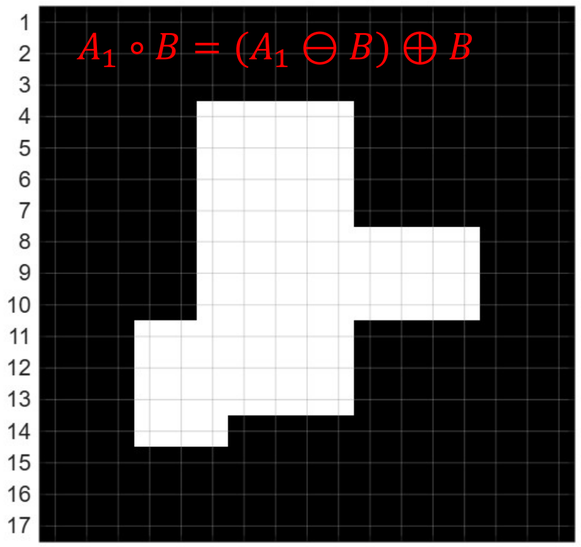


Esqueletizacao - Iteracao k = 1

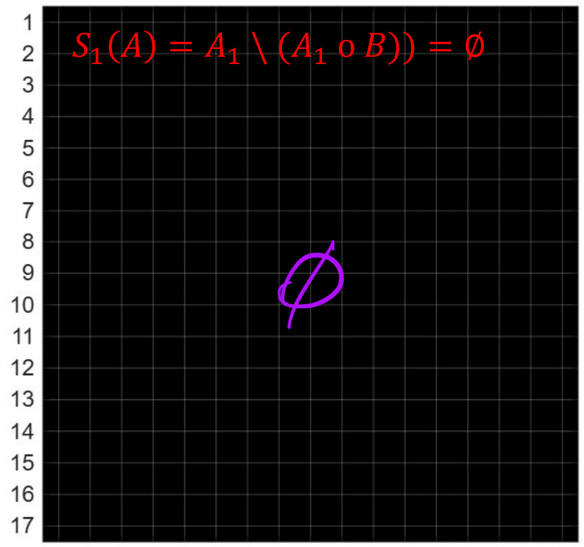
Erosao: A_1



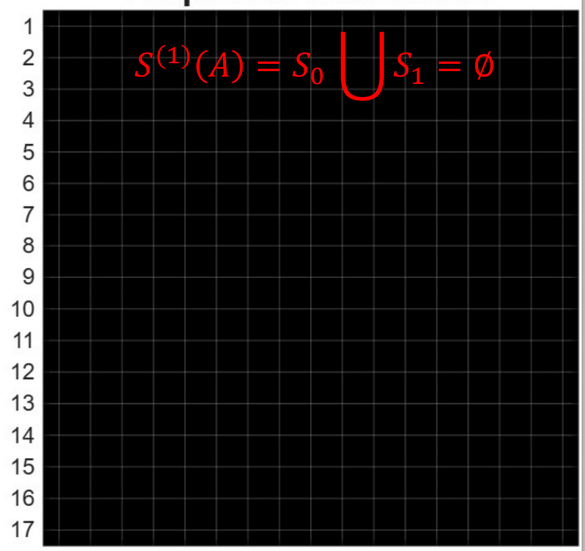
Abertura: A_1 o B



Diferenca: S_1

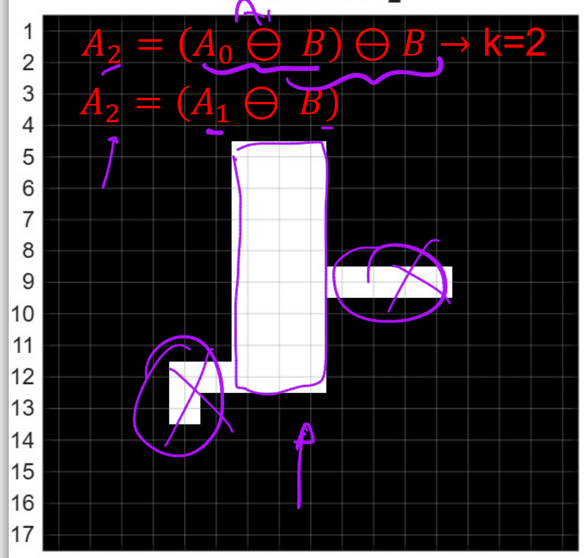


Esqueleto acumulado

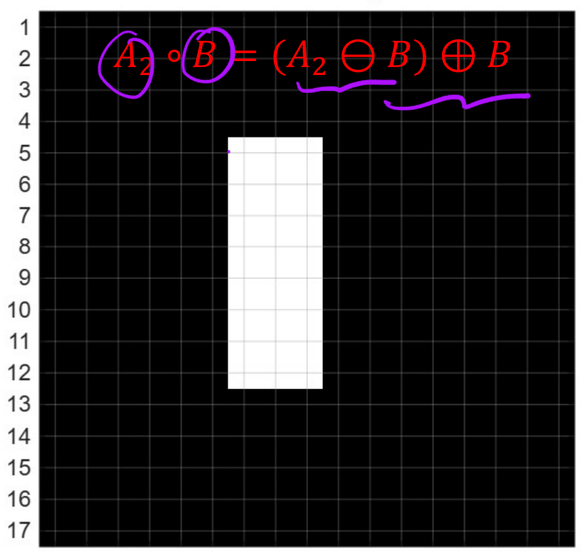


Esqueletização - Iteração k = 2

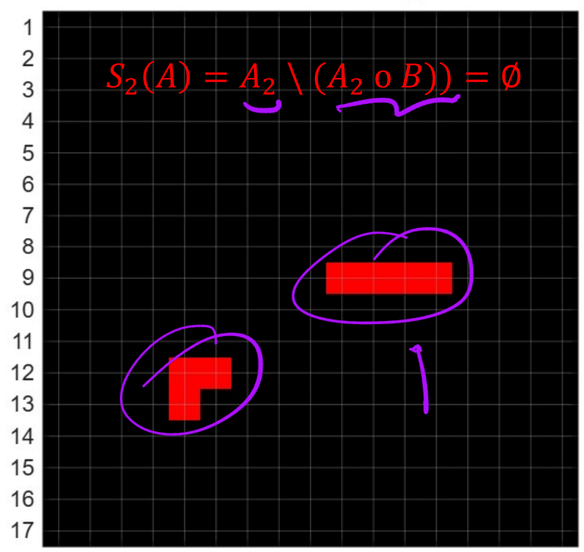
Erosão: A_2



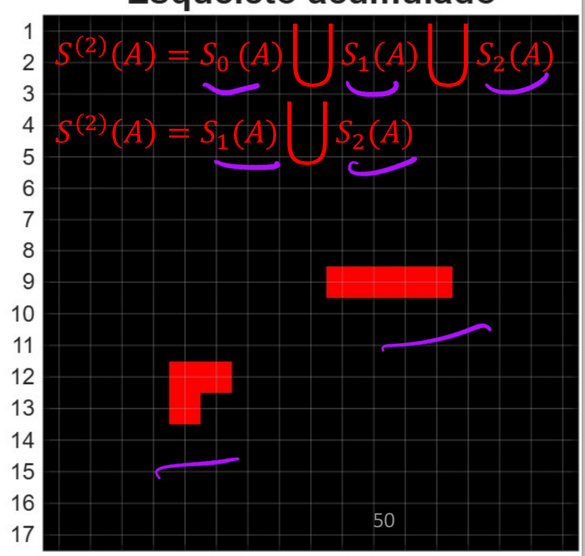
Abertura: A_2 ° B



Diferença: S_2

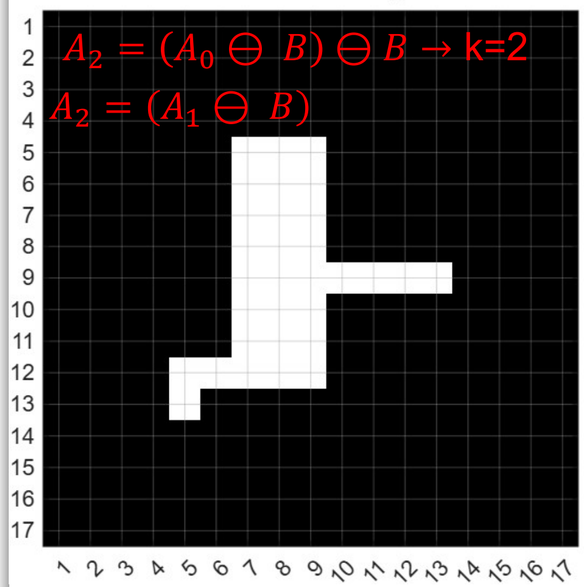


Esqueleto acumulado

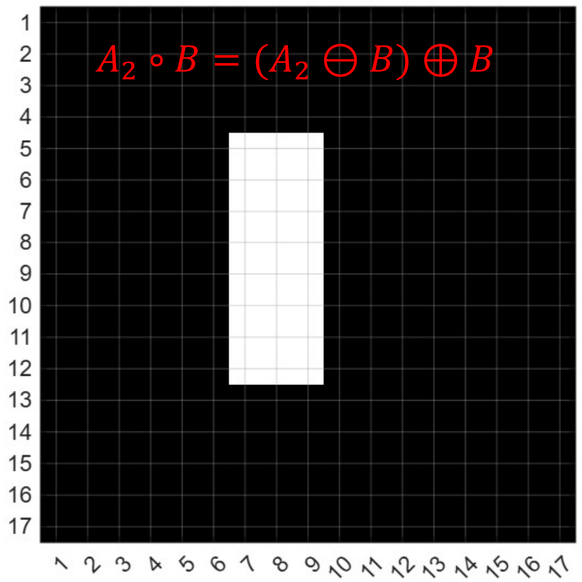


Esqueletização - Iteração k = 2

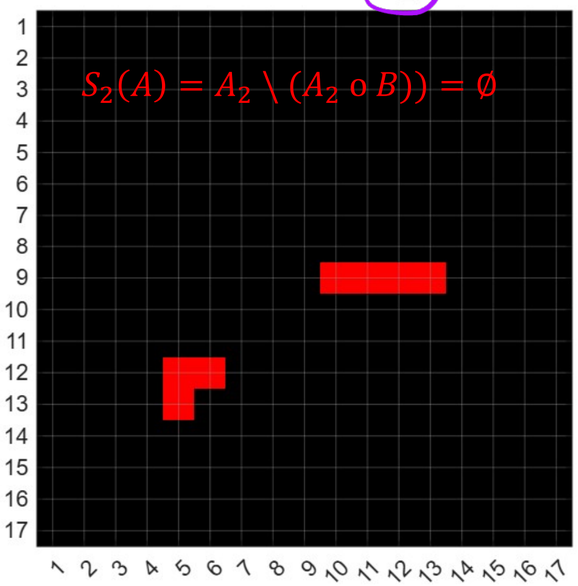
Erosão: A_2



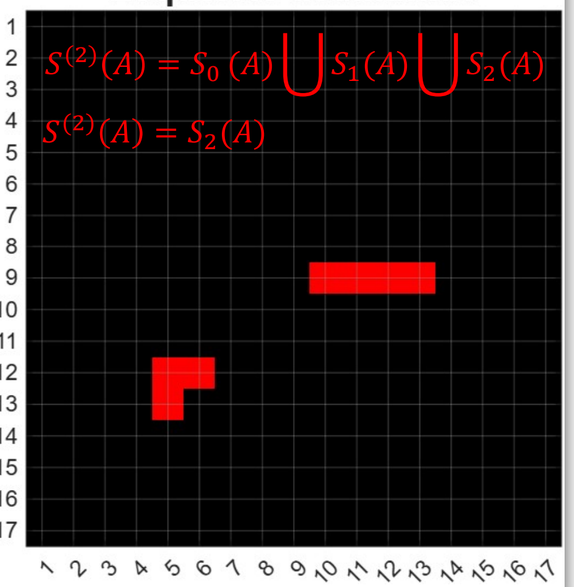
Abertura: A_2 ° B



Diferença: S_2

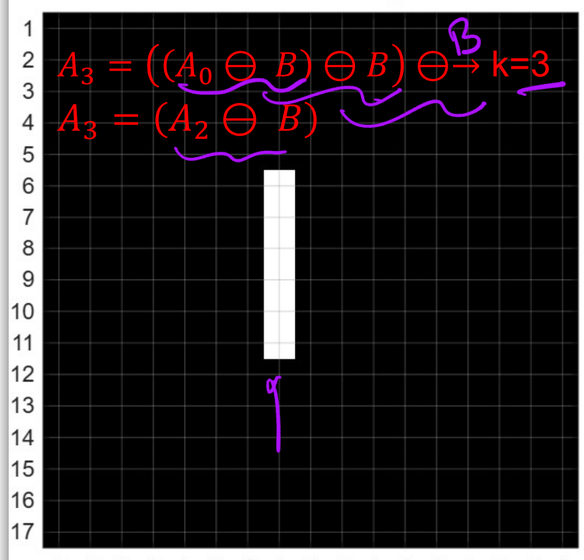


Esqueleto acumulado

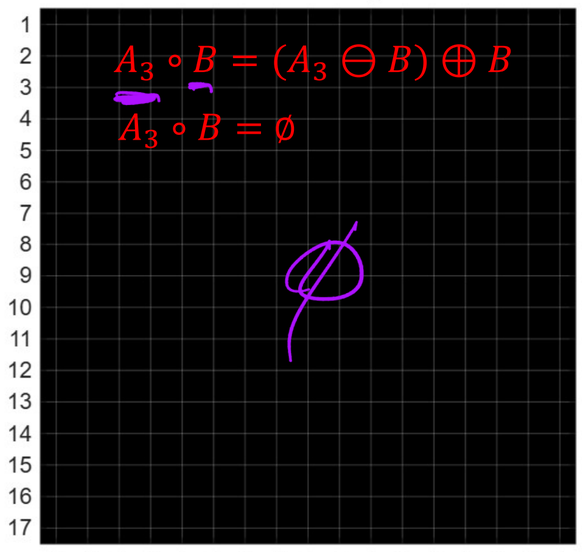


Esqueletização - Iteração k = 3

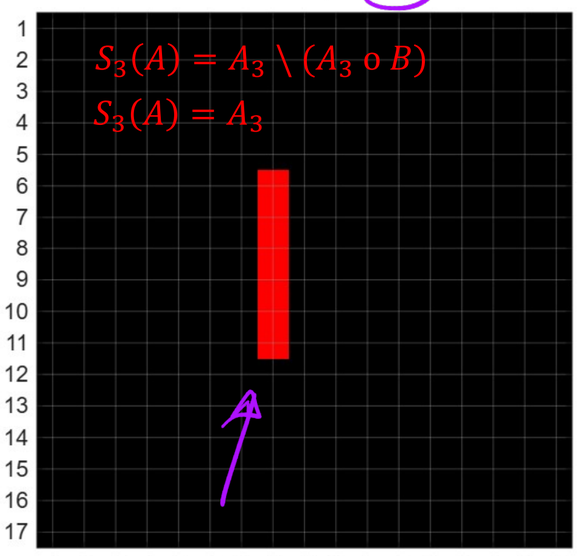
Erosão: A_3



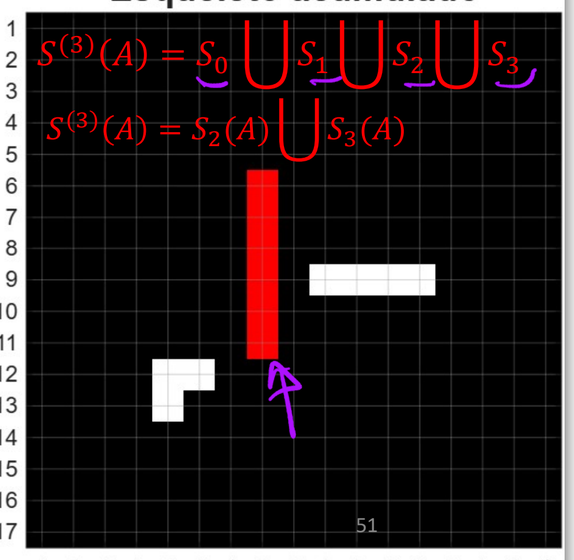
Abertura: A_3 ° B

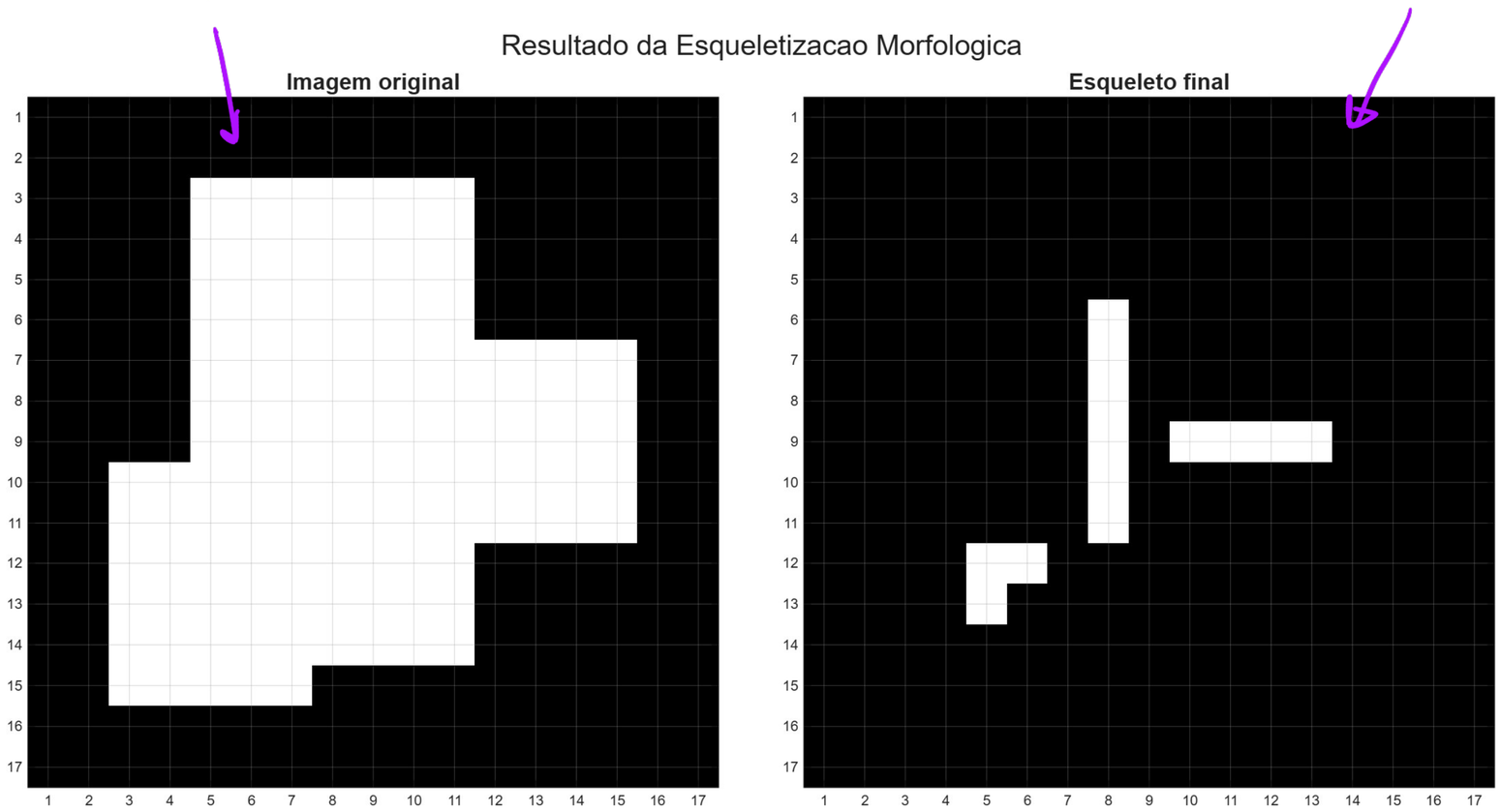


Diferença: S_3



Esqueleto acumulado





Considerações:

1. O objeto original é conectado, enquanto o esqueleto final possui três componentes desconectados.

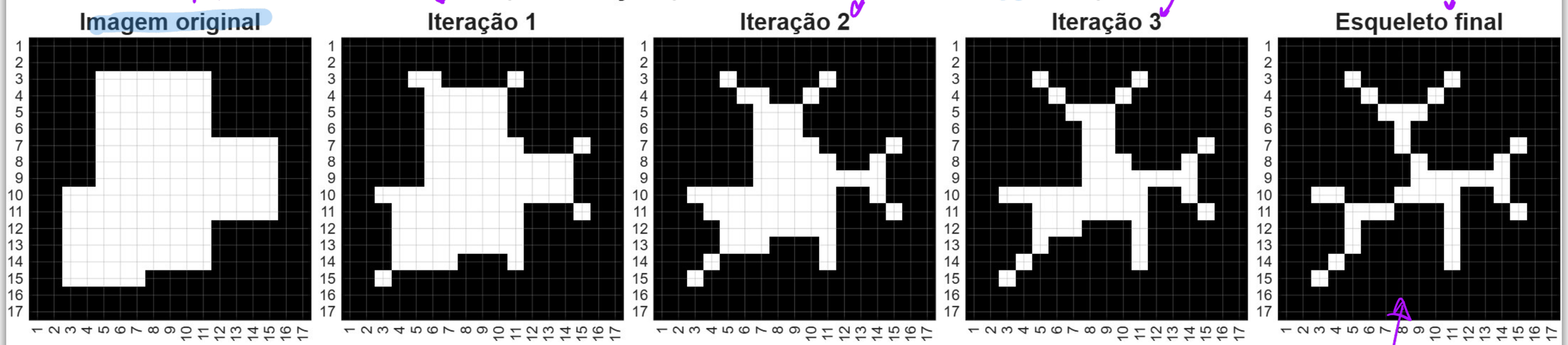
2. Isso não é um erro. A esqueletização por erosões e aberturas sucessivas, utilizando um elemento estruturante fixo, não garante a preservação da conectividade do objeto original.

Existem outros algoritmos de esqueletização, baseados em afinamento iterativo (*thinning*), que procuram preservar a conectividade do objeto durante a redução de sua espessura.

esqueletização c/ bwmorph

A

Esqueletização por afinamento iterativo - bwmorph



Sfinal = bwmorph(A, 'skel', Inf);

O algoritmo bwmorph examina as configurações locais de pixels e aplica regras iterativas de remoção, procurando reduzir a espessura do objeto sem romper sua conectividade.

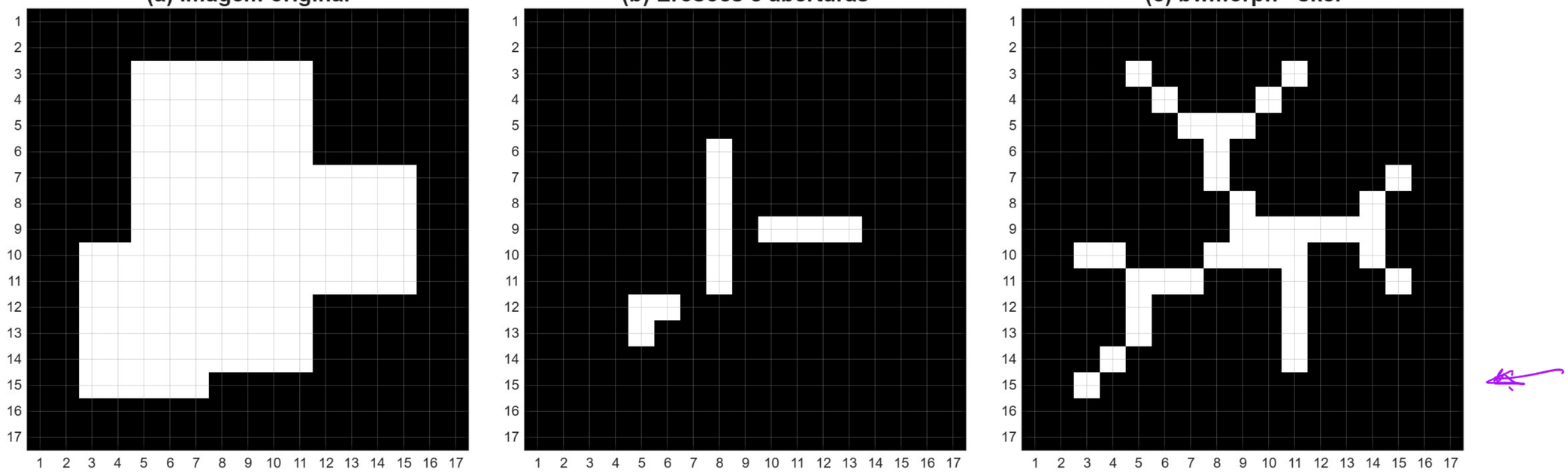
A → imagem de entrada

'skel' → operação morfológica de esqueletização (*skeletonization*)

Inf → repete a operação até que a imagem não apresente mais alterações

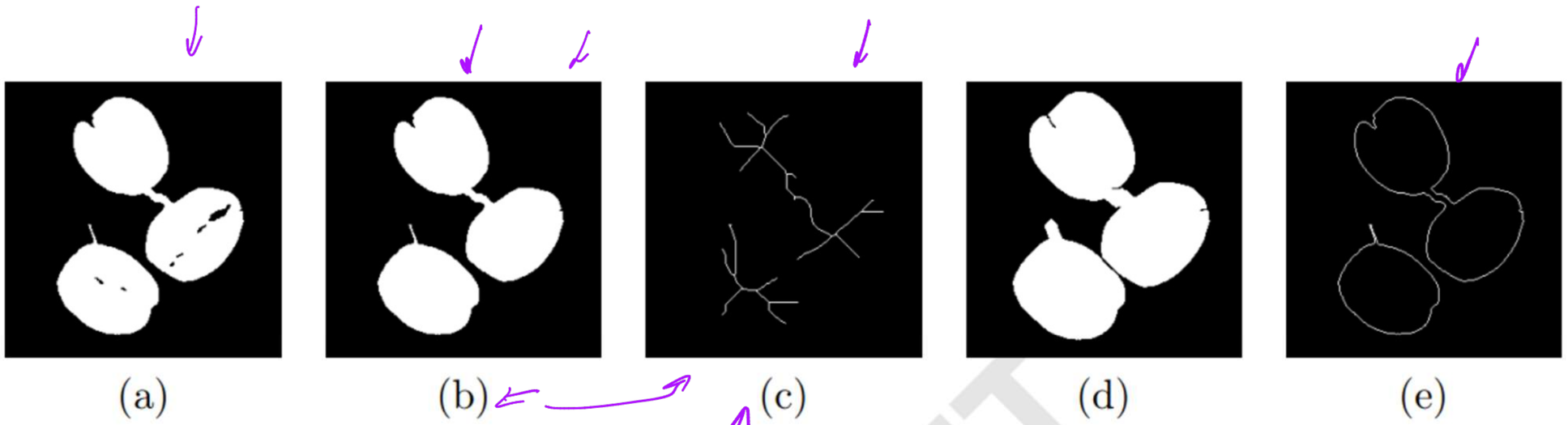
Sfinal → imagem binária resultante, contendo o esqueleto do objeto

Comparação dos métodos de esqueletização



Característica	Erosões e aberturas	bwmorph(A, 'skel', Inf);
Esqueleto binário	sim	Sim
Preservação da conectividade	Não garantida	Preservada pelo afinamento
Informação da espessura original	Preservada quando armazenados os conjutos S_k e seus níveis de k	Não está disponível no esqueleto binário
Reconstrução exata	Possível com S_k, k e B	Não garantida
Método	Erosões e aberturas	Afinamento iterativo

Existem muitos tipos de operações morfológicas...



(a) (b) (c) (d) (e)

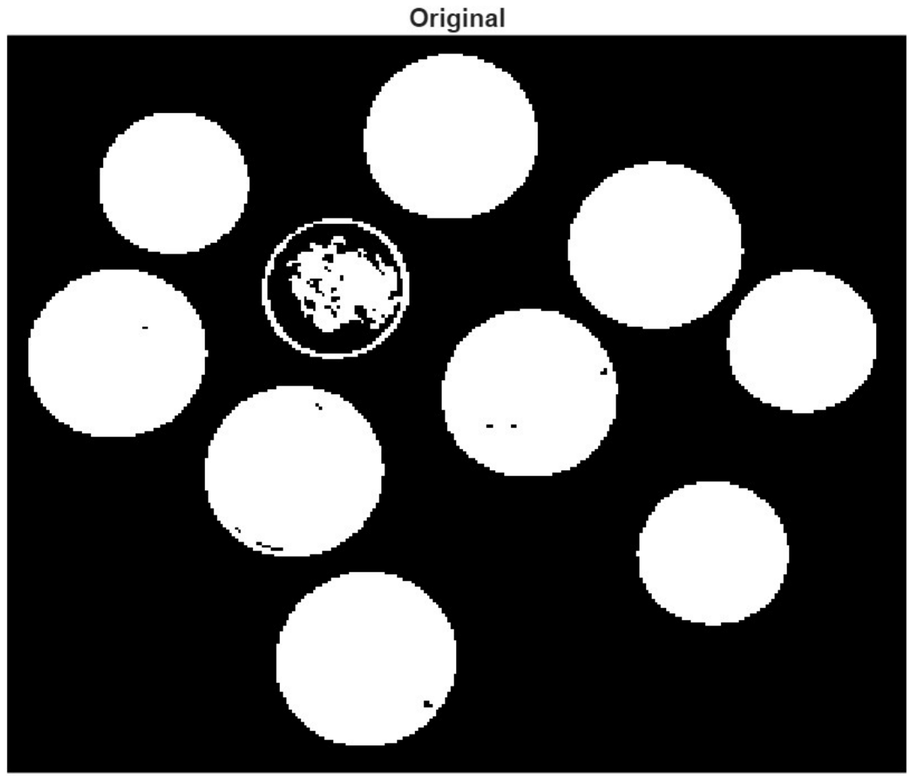
Figure 24.7: Morphological techniques obtained with `imfill`, `bwmorph` and `bwperim` functions. (a) Original image, (b) original image with filled holes, (c) skeleton of (b), (d) thickening of (b), (e) perimeter extraction of (b).

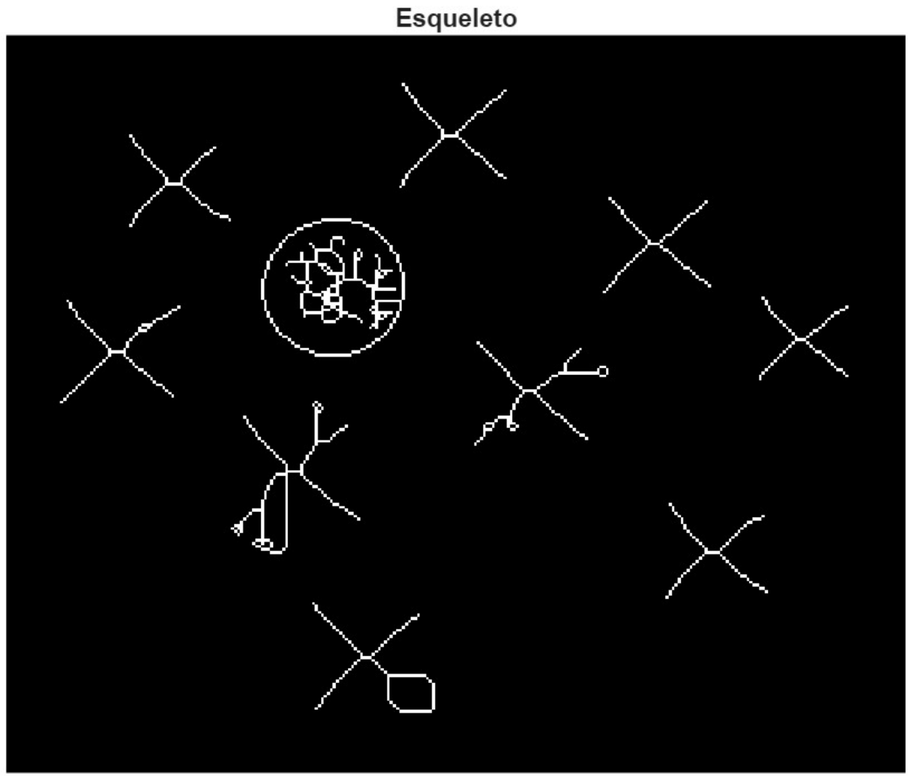
```
BW_skel = bwmorph(BW, 'skel', Inf);
```

Esqueletização (skeleton) PULAR

Esqueletização reduz objetos à sua estrutura central preservando a conectividade, mas sensível a ruído

O algoritmo remove iterativamente pixels das bordas preservando a conectividade.”

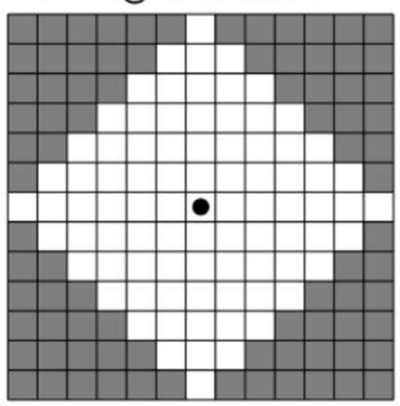




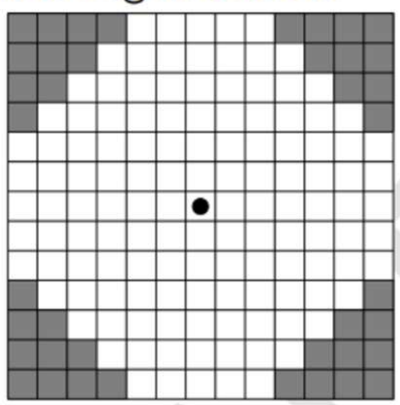

```
clc, clear, close all
BW = imbinarize(imread('coins.png'));
skel = bwmorph(BW, 'skel', Inf);
figure
subplot(1,2,1), imshow(BW), title('Original')
subplot(1,2,2), imshow(skel), title('Esqueleto')
```

O elemento estruturante sempre é configurável

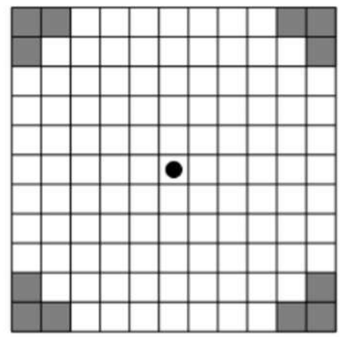
```
r = 6;
se = strel('diamond',r);
se.Neighborhood
```



```
r = 6;
se = strel('octagon',r);
se.Neighborhood
```

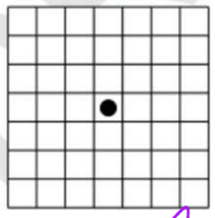


```
r = 6;
se = strel('disk',r);
se.Neighborhood
```

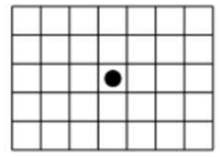


```
len = 7; deg = 0;
se = strel('line',len,deg);
se.Neighborhood
```

```
w = 7;
se = strel('square',w);
se.Neighborhood
```



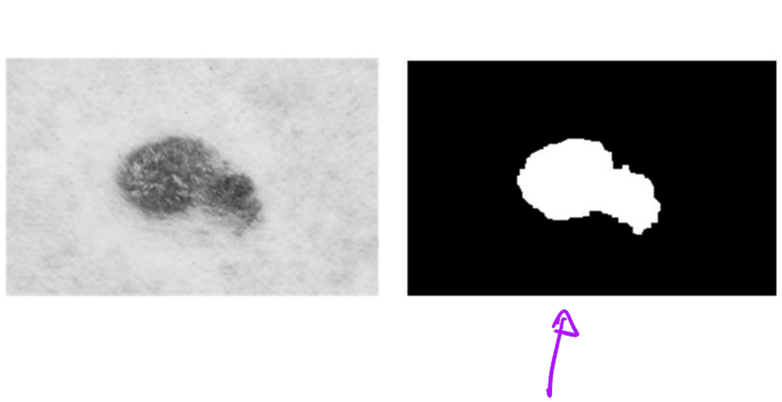
```
m = 5; n = 7;
se = strel('rectangle',[m n]);
se.Neighborhood
```

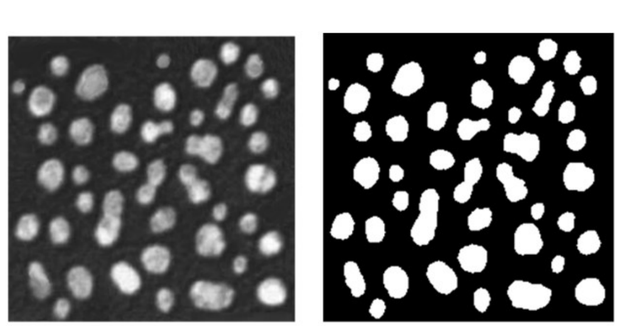


DRAFT

Rotulação de regiões binárias (labeling)

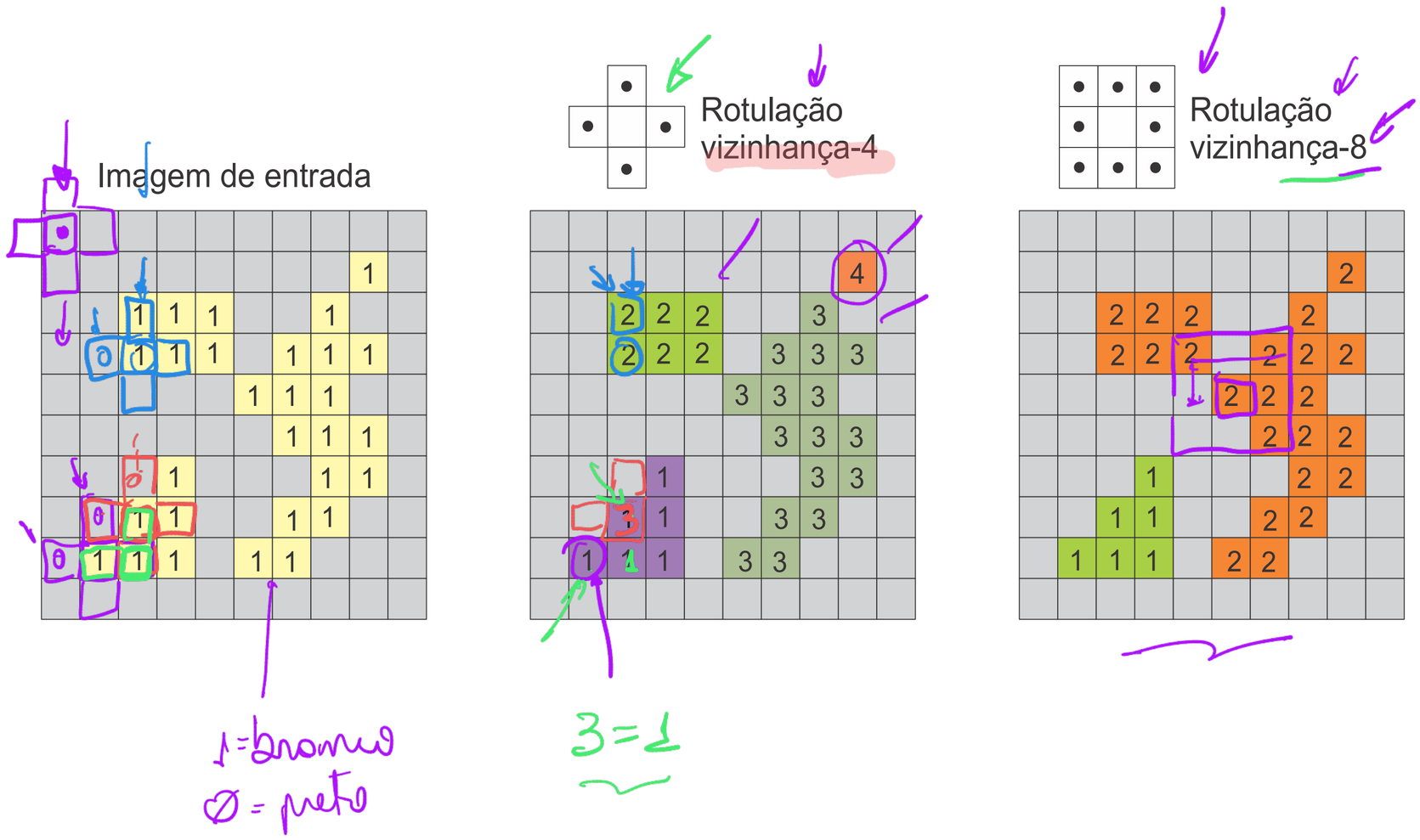
- Quantas lesões de pele tem na imagem?
- Quantas células tem na imagem?





Rotulação de regiões binárias (labeling)

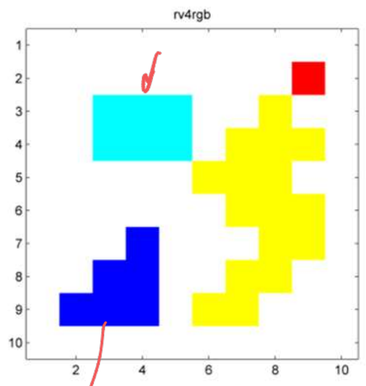
O objetivo do labeling é atribuir um label para cada objeto da imagem binária, isto é, *discriminar* cada objeto.

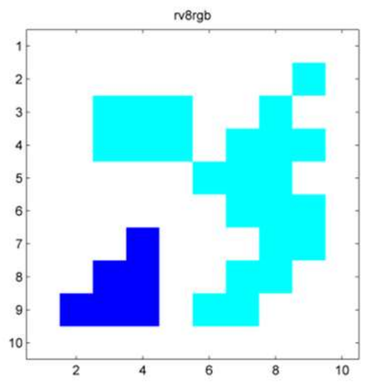


Rotulação de regiões binárias (labeling)

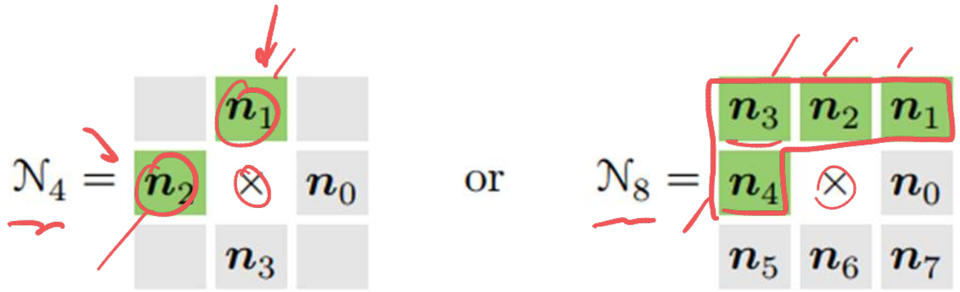
```
clear all, close all
clc

bw = [ 0 0 0 0 0 0 0 0 0 0 0 0
       0 0 0 0 0 0 0 0 0 1 0 0
       0 0 1 1 1 0 0 1 0 0 0 0
       0 0 1 1 1 0 1 1 1 0 0 0
       0 0 0 0 0 1 1 1 0 0 0 0
       0 0 0 0 0 0 1 1 1 1 0 0
       0 0 0 1 0 0 0 1 1 1 0 0
       0 0 1 1 0 0 1 1 0 0 0 0
       0 1 1 1 0 1 1 0 0 0 0 0
       0 0 0 0 0 0 0 0 0 0 0 0 ];

bw = logical(bw);

% A saída de bwlabel
% é da classe double
[rv4 nrv4] = bwlabel(bw, 4);
[rv8 nrv8] = bwlabel(bw, 8);

% Aplica pseudocores na
% imagem rotulada para
% visualização
rv4rgb = label2rgb(rv4);
rv8rgb = label2rgb(rv8);
disp(['n. obj em rv4 = ',...
      num2str(nrv4)])
disp(['n. obj em rv8 = ',...
      num2str(nrv8)])

% Display
figure, image(rv4rgb)
title('rv4rgb')
figure, image(rv8rgb)
title('rv8rgb')
```

```
>> rv4
rv4 =
     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     4     0
     0     0     2     2     2     0     0     3     0     0
     0     0     2     2     2     0     3     3     3     0
     0     0     0     0     0     3     3     3     0     0
     0     0     0     0     0     0     3     3     3     0
     0     0     0     1     0     0     0     3     3     0
     0     0     1     1     0     0     3     3     0     0
     0     1     1     1     0     3     3     0     0     0
     0     0     0     0     0     0     0     0     0     0
>> rv8
rv8 =
     0     0     0     0     0     0     0     0     0     0
     0     0     0     0     0     0     0     0     2     0
     0     0     2     2     2     0     0     2     0     0
     0     0     2     2     2     0     2     2     2     0
     0     0     0     0     0     2     2     2     0     0
     0     0     0     0     0     0     2     2     2     0
     0     0     0     1     0     0     0     2     2     0
     0     0     1     1     0     0     2     2     0     0
     0     1     1     1     0     2     2     0     0     0
     0     0     0     0     0     0     0     0     0     0
```





Rotulação de regiões binárias (labeling)



Início do pass 1 (union ou label propagation)

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figura: Wilhelm Burger and Mark J. Burge, Digital Image Processing: An Algorithmic Introduction, 3rd ed. Springer, 2022. Site do livro: https://imagingbook.com/

Encontrei objeto e todos os vizinhos são fundo

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	1	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

→

Novo label (label inicial = 2)

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	1	0	0	1	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Encontrei objeto e algum vizinho não é fundo

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	1	0	0	1	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

→

Propago o menor label. Não houve colisão

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	1	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Encontrei objeto e todos os vizinhos são fundo

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	1	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

→

Novo label

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Encontrei objeto e algum vizinho não é fundo

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	1	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

→

Propago o menor label. Não houve colisão

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	1	0
0	1	1	1	1	1	1	0	0	1	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Encontrei objeto e algum vizinho não é fundo

0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	2	2	0	0	3	3	0	4	0	
0	5	5	5	1	1	1	0	0	1	0	0	1	0	
0	0	0	0	1	0	1	0	0	0	0	0	1	0	
0	1	1	1	1	1	1	1	1	1	1	1	1	0	
0	0	0	0	1	1	1	1	1	1	1	1	1	0	
0	1	1	0	0	0	1	0	1	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	1	1	0	0	0	→
0	0	0	0	0	1	1	0	0	0	1	0	1	0	
0	1	1	1	1	1	1	0	0	0	0	0	1	0	
0	0	0	0	1	0	1	0	0	0	0	0	1	0	
0	1	1	1	1	1	1	1	1	1	1	1	1	0	
0	0	0	0	1	1	1	1	1	1	1	1	1	0	
0	0	0	0	0	0	1	0	1	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Propago o menor label. Houve colisão entre labels 5 e 2

0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	0	0	0	2	2	0	0	3	3	0	4	0	
0	5	5	5	2	1	1	0	0	1	0	0	1	0	
0	0	0	0	1	0	1	0	0	0	0	0	1	0	
0	1	1	1	1	1	1	1	1	1	1	1	1	0	
0	0	0	0	1	1	1	1	1	1	1	1	1	0	
0	1	1	0	0	0	1	0	1	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	1	1	0	0	0	
0	0	0	0	0	1	1	0	0	0	1	0	1	0	
0	1	1	1	1	1	1	0	0	0	0	0	1	0	
0	0	0	0	1	0	1	0	0	0	0	0	1	0	
0	1	1	1	1	1	1	1	1	1	1	1	1	0	
0	0	0	0	1	1	1	1	1	1	1	1	1	0	
0	0	0	0	0	0	1	0	1	0	0	0	0	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	

vetor PARENT

0	0	0	0	2	0	0	0	0	0	0	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

Encontrei objeto e algum vizinho não é fundo

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

→

Propago o menor label. Houve colisão entre labels 6 e 2

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	0	2	2	0	0	0	0	0	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

Propago o menor label. Houve colisão entre labels 4 e 2

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	1	1	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	1	1	0	0	0
0	0	0	0	0	1	1	0	0	0	1	0	1	0
0	1	1	1	1	1	1	0	0	0	0	0	1	0
0	0	0	0	1	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	2	2	2	0	0	0	0	0	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

Propago o menor label. Houve colisão entre labels 10 e 8

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	0	1	0	0	0	0	0	1	0
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	2	2	2	0	0	0	8	0	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

Propago o menor label. Houve colisão entre labels 11 e 8

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	0	8	0	0	0	0	0	9	0
0	11	11	8	1	1	1	1	1	1	1	1	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	2	2	2	0	0	0	8	8	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

Propago o menor label. Houve colisão entre labels 9 e 8

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	0	8	0	0	0	0	0	9	0
0	11	11	8	8	8	8	8	8	8	8	8	1	0
0	0	0	0	1	1	1	1	1	1	1	1	1	0
0	0	0	0	0	0	1	0	1	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	2	2	2	0	0	8	8	8	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

Fim do pass 1 (union ou label propagation)

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	0	8	0	0	0	0	0	9	0
0	11	11	8	8	8	8	8	8	8	8	8	8	0
0	0	0	0	8	8	8	8	8	8	8	8	8	0
0	0	0	0	0	0	8	0	8	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	2	2	2	0	0	8	8	8	0	0	...
1	2	3	4	5	6	7	8	9	10	11	12	13	

Início do pass 2 (find ou collision solving)

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	4	0
0	5	5	5	2	2	2	0	0	3	0	0	4	0
0	0	0	0	2	0	2	0	0	0	0	0	4	0
0	6	6	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	9	0
0	10	10	10	8	8	8	0	0	0	0	0	9	0
0	0	0	0	8	0	8	0	0	0	0	0	9	0
0	11	11	8	8	8	8	8	8	8	8	8	8	0
0	0	0	0	8	8	8	8	8	8	8	8	8	0
0	0	0	0	0	0	8	0	8	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

0	0	0	2	2	2	0	0	8	8	8	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

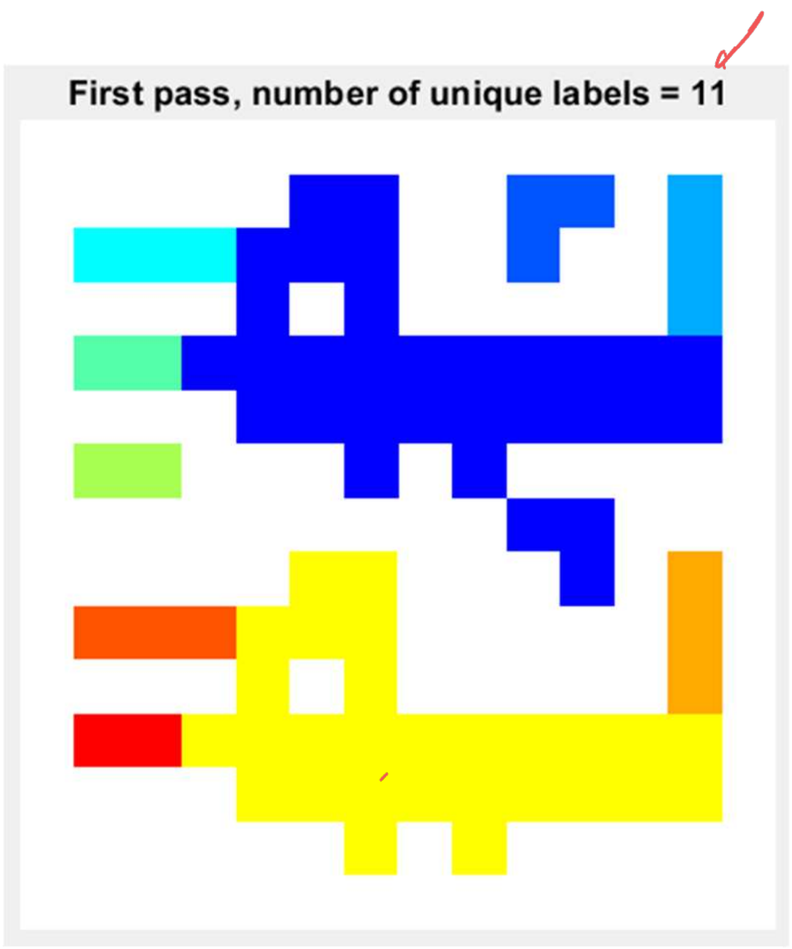
Atualizo todos os labels com j. while(PARENT(j) ≠ 0) {j = PARENT(j)}

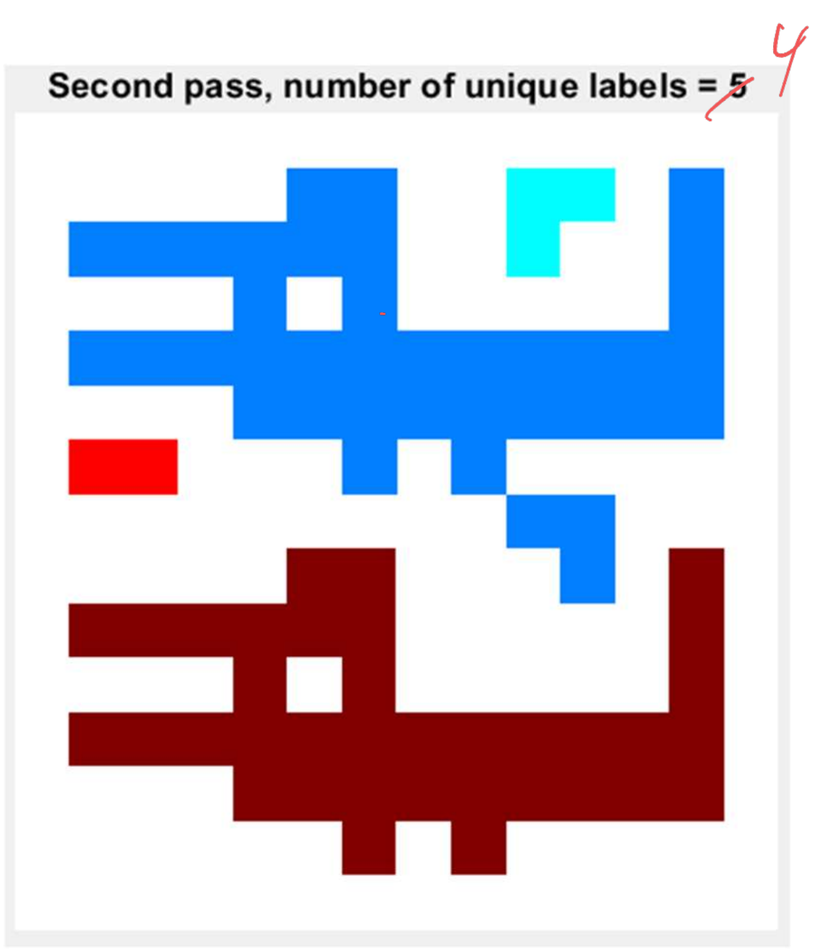
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	2	2	0	0	3	3	0	2	0
0	2	2	2	2	2	2	0	0	3	0	0	2	0
0	0	0	0	2	0	2	0	0	0	0	0	2	0
0	2	2	2	2	2	2	2	2	2	2	2	2	0
0	0	0	0	2	2	2	2	2	2	2	2	2	0
0	7	7	0	0	0	2	0	2	0	0	0	0	0
0	0	0	0	0	0	0	0	0	2	2	0	0	0
0	0	0	0	0	8	8	0	0	0	2	0	8	0
0	8	8	8	8	8	8	0	0	0	0	0	8	0
0	0	0	0	8	0	8	0	0	0	0	0	8	0
0	8	8	8	8	8	8	8	8	8	8	8	8	0
0	0	0	0	8	8	8	8	8	8	8	8	8	0
0	0	0	0	0	0	8	0	8	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0

vetor PARENT

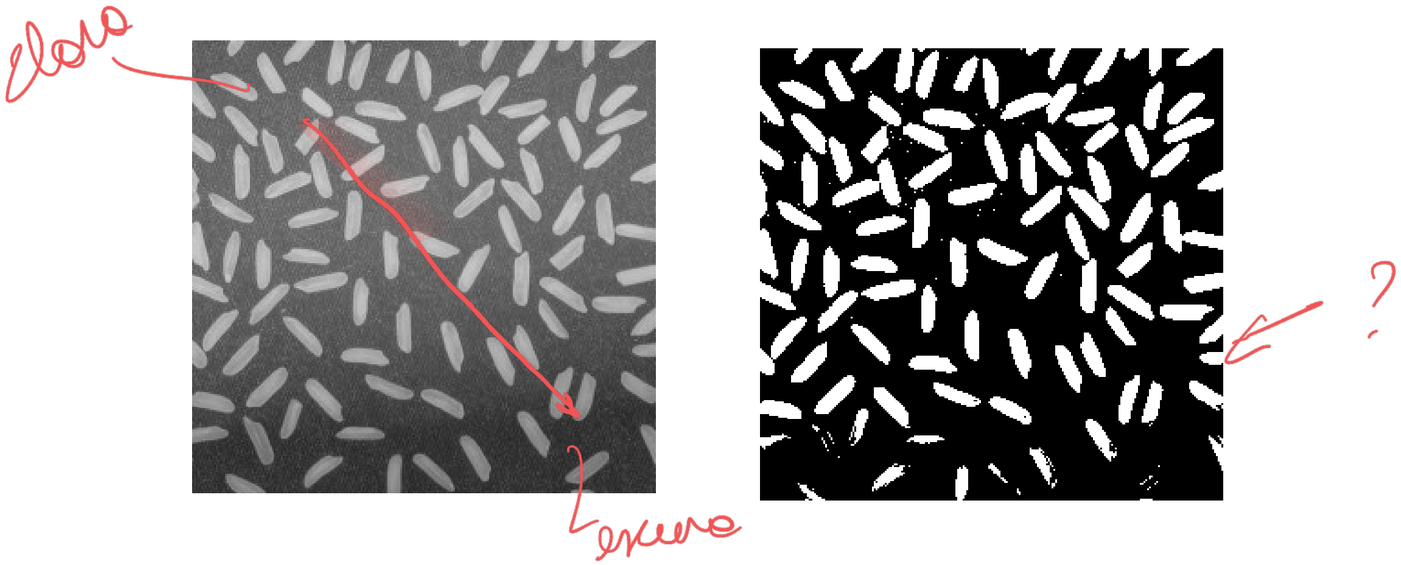
0	0	0	2	2	2	0	0	8	8	8	0	0	…
1	2	3	4	5	6	7	8	9	10	11	12	13	

- Implementação do deste algoritmo disponível no material de aula, arquivo *unionfind_labeling_v251.m*





- A segmentação por limiarização global pode ser comprometida em situações nas quais a iluminação na imagem não é uniforme (homogênea):

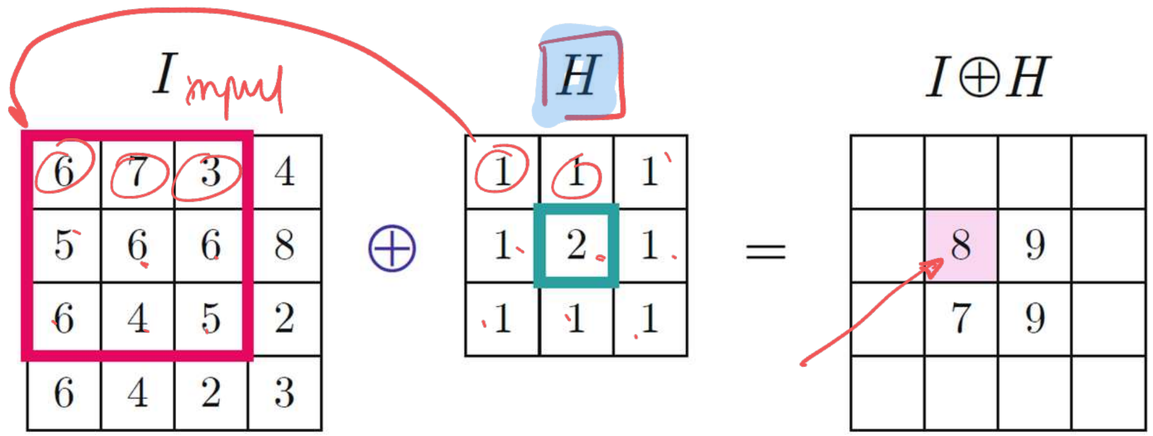


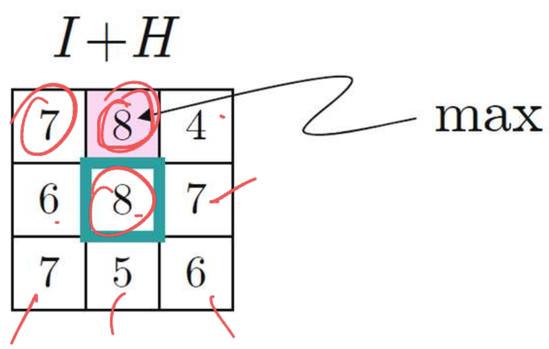
- Existem diferentes abordagens para a solução. Por exemplo:
  - Transformada top-hat
  - Limiarização local (adaptativa)

Imagem: https://www.mathworks.com/help/images/correcting-nonuniform-illumination.html

Dilatação grayscale

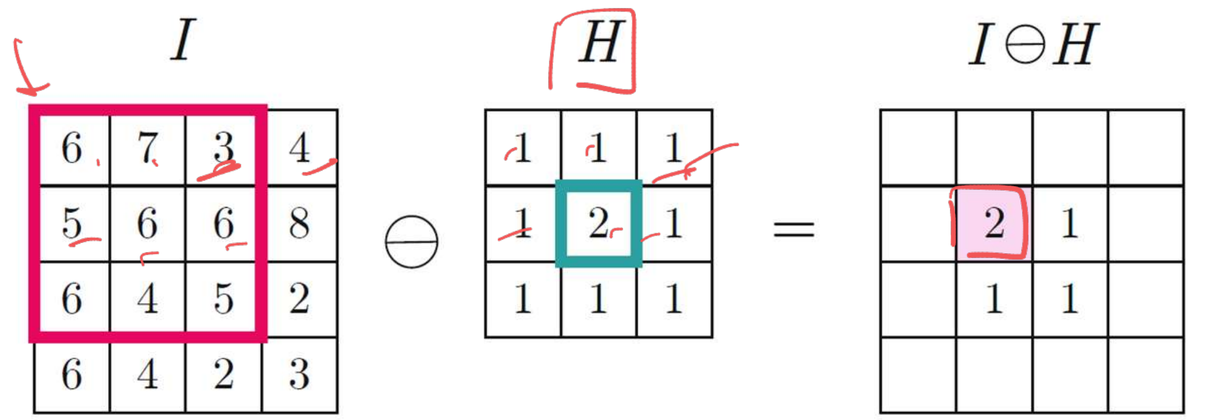
$$(I \oplus H)(u,v) = \max_{(i,j)\in H} \left(I(u+i, v+j) + H(i,j)\right)$$

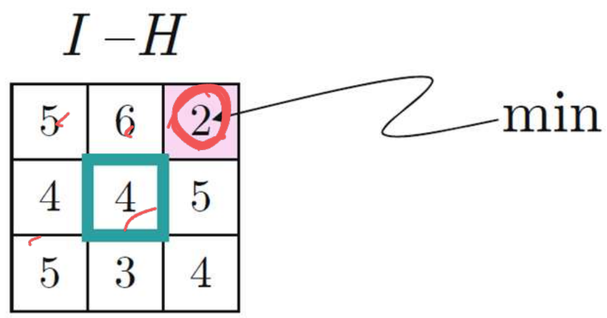


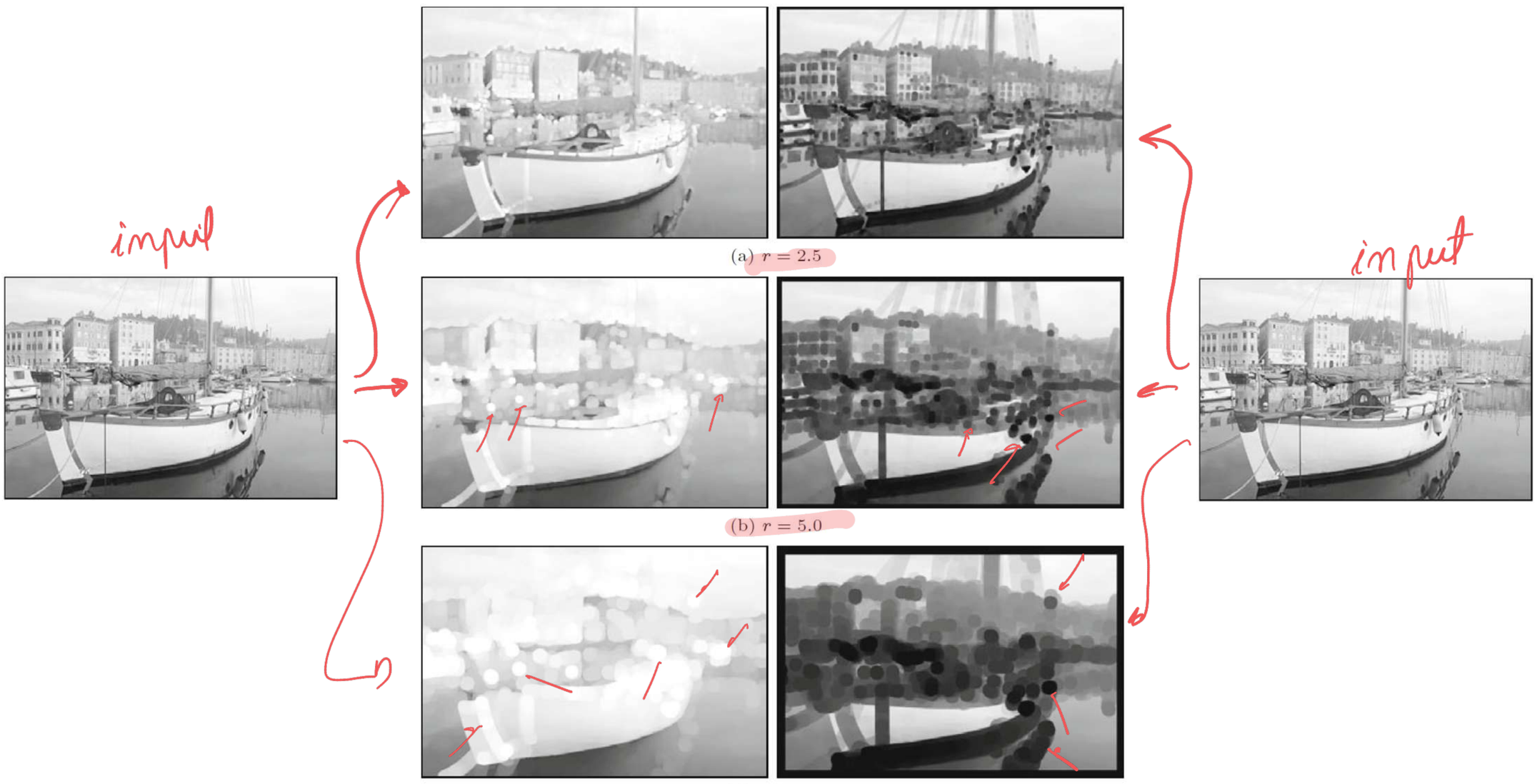


Erosão grayscale

$$(I \ominus H)(u,v) = \min_{(i,j)\in H} \big(I(u+i, v+j) - H(i,j)\big)$$







(a) $r = 2.5$

(b) $r = 5.0$

(c) $r = 10.0$

Elemento estruturante circular de raio r.
O aumento de r amplia a vizinhança utilizada nas operações de erosão e dilatação em tons de cinza.”

Transformada top-hat: Correção de iluminação

- A transformada top-hat pode ser utilizada como uma etapa de pré-processamento da imagem, **para uniformizar a iluminação do fundo** (background), antes de aplicar a limiarização global.

- A transformada top-hat TopHat de uma imagem grayscale G é:

$$\text{TopHat} = G - (G \circ S)$$

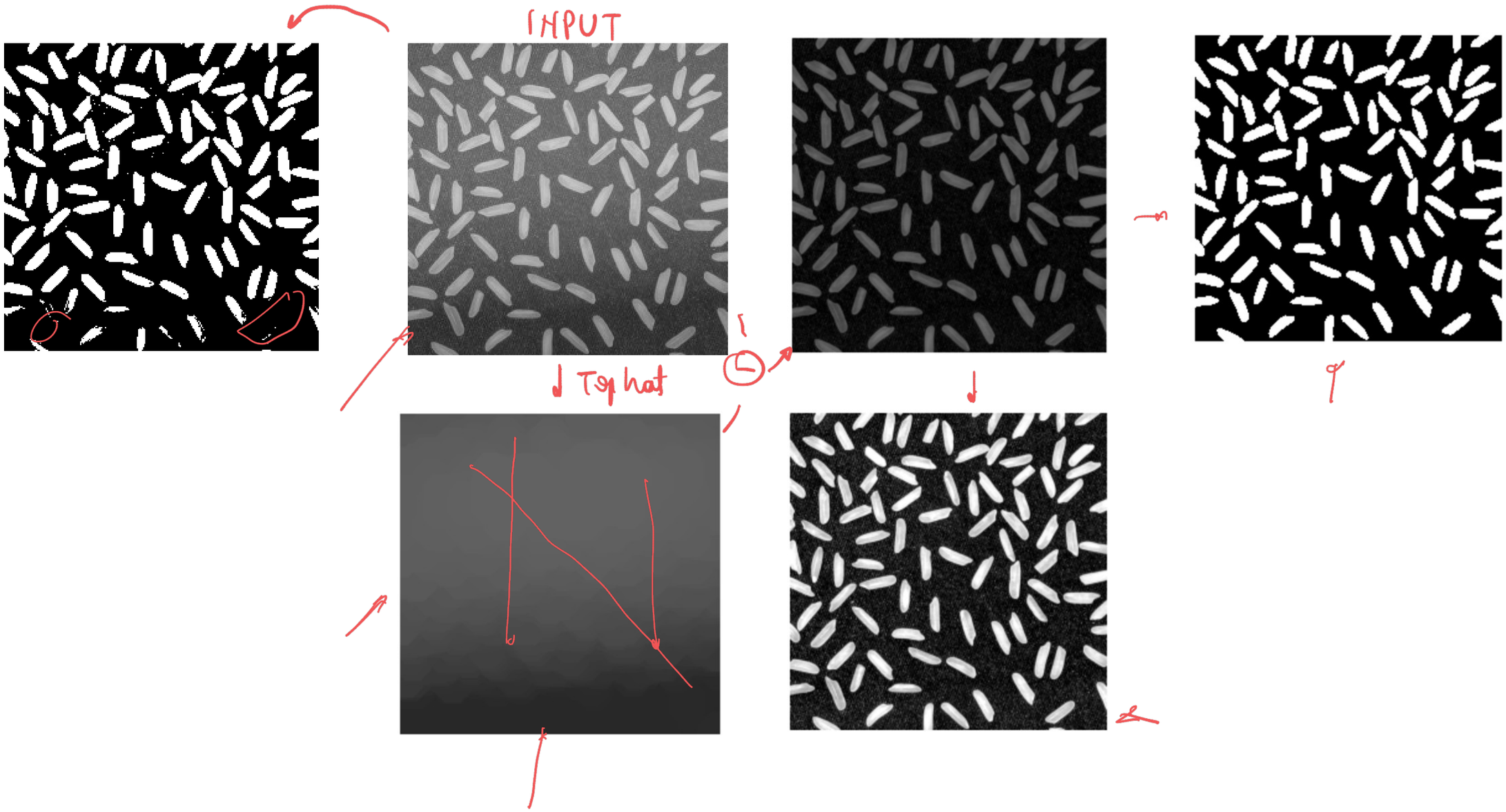
onde '$\circ$' é a operação morfológica de abertura de G por um elemento estruturante S e '$-$' é uma subtração de imagens.

- Assim como nas operações morfológicas binárias, a abertura morfológica de uma imagem grayscale **$(G \circ S)$** é o resultado da **erosão seguida da dilatação**, isto é, aplicar a erosão na imagem G usando S e depois aplicar a dilatação na imagem erodida usando S.

Ver:
https://www.mathworks.com/help/images/correcting-nonuniform-illumination.html

Transformada top-hat

https://www.mathworks.com/help/images/correcting-nonuniform-illumination.html



Th local algoritmo 1

https://www.mathworks.com/help/images/ref/adaptthresh.html

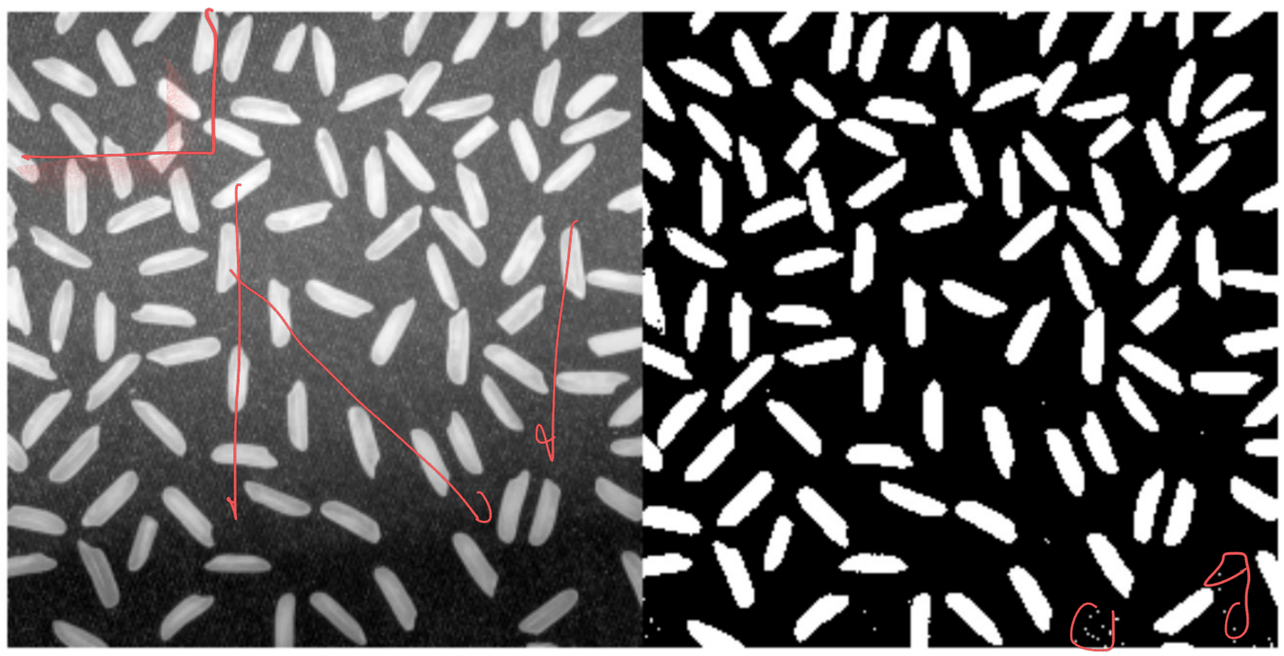
calcular, para cada pixel, a média em uma vizinhança *sxs* e realizar seguinte operação:
- se o valor do pixel é t porcento menor que a média, ele é considerado '0'
- caso contrário, é considerado '1'

$$f(i,j) < (1-t) \cdot \mu(i,j) \Rightarrow 0$$

$$f(i,j) > (1-t) \cdot \mu(i,j) \Rightarrow 1$$

$\mu(i,j)$ é a média da região SxS

Ou seja, o limiar não é fixo — ele é proporcional à **iluminação local**.



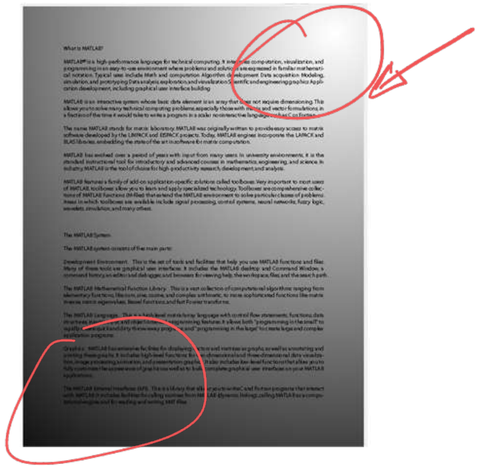
Isso é basicamente o que o **adaptthresh**() faz:

```
T = adaptthresh(I, 0.5); % sensibilidade
BW = imbinarize(I, T);
```

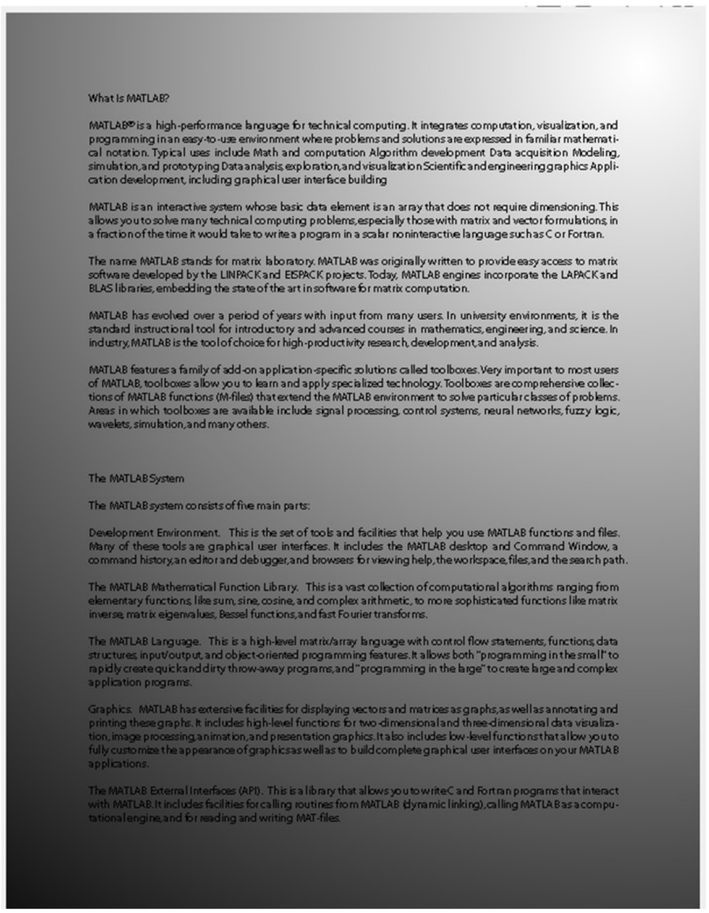
Th local algoritmo 2

```
% adaptive_Th [script]
% Tutorial 'Adaptive Thresholding',
% capítulo 15, páginas 350 até 352 do livro
% Oge Marques, Practical image and video
% processing using MATLAB, Wiley, 2011.
clear all; close all; clc

g = imread('gradient with text.tif');
figure, imshow(g), title('in')

% Global Th
ggth = im2bw(g, graythresh(g));
figure, imshow(ggth), title('ggth')
figure, imhist(g) % otsu fail
% Adaptive Th
% Função blockproc é sem overlap
% Cria function handle
fun = @(myBlock) adaptThStd(myBlock.data);
gath = blockproc(g, [10 10], fun); % processa
figure, imshow(gath), title(gath)
```

```
function y = adaptThStd(x)
% x: bloco da imagem (subimagem)
% y: bloco processado

% std baixo -> é fundo
if(std2(x)) < 1
    % Sai um bloco de uns
    y = ones(size(x,1),size(x,2));
% std alto -> é texto
else
    % Aplica Otsu no bloco
    y = im2bw(x, graythresh(x));
end
```



Th local algoritmo 2



What Is MATLAB?

MATLAB® is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical uses include Math and computation Algorithm development Data acquisition Modeling, simulation, and prototyping Data analysis, exploration, and visualization Scientific and engineering graphics Application development, including graphical user interface building

MATLAB is an interactive system whose basic data element is an array that does not require dimensioning. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a scalar noninteractive language such as C or Fortran.

The name MATLAB stands for matrix laboratory. MATLAB was originally written to provide easy access to matrix software developed by the LINPACK and EISPACK projects. Today, MATLAB engines incorporate the LAPACK and BLAS libraries, embedding the state of the art in software for matrix computation.

MATLAB has evolved over a period of years with input from many users. In university environments, it is the standard instructional tool for introductory and advanced courses in mathematics, engineering, and science. In industry, MATLAB is the tool of choice for high-productivity research, development, and analysis.

MATLAB features a family of add-on application-specific solutions called toolboxes. Very important to most users of MATLAB, toolboxes allow you to learn and apply specialized technology. Toolboxes are comprehensive collections of MATLAB functions (M-files) that extend the MATLAB environment to solve particular classes of problems. Areas in which toolboxes are available include signal processing, control systems, neural networks, fuzzy logic, wavelets, simulation, and many others.

The MATLAB System

The MATLAB system consists of five main parts:

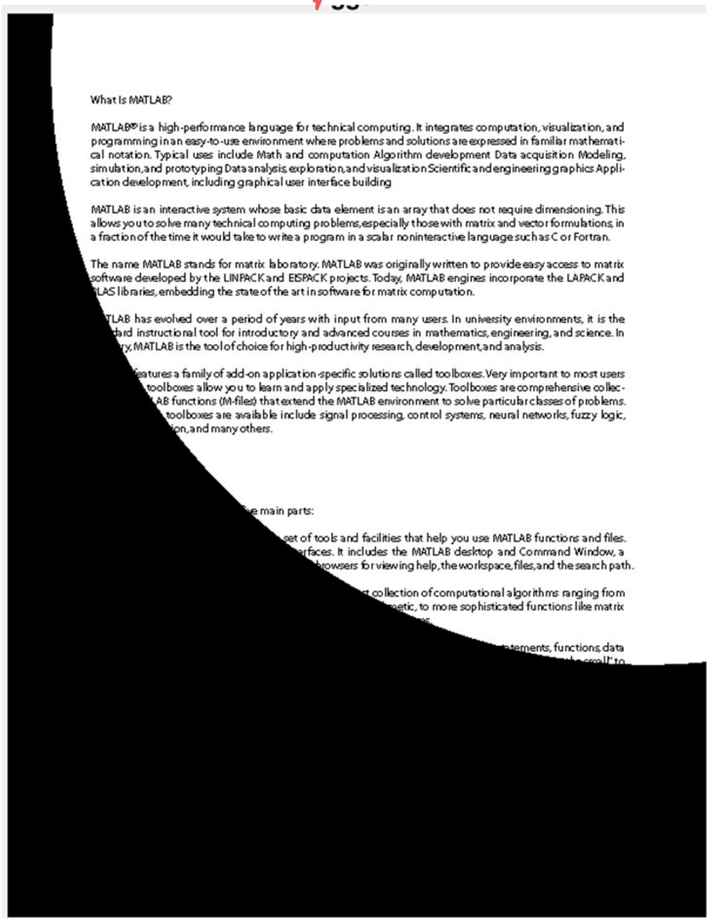
Development Environment. This is the set of tools and facilities that help you use MATLAB functions and files. Many of these tools are graphical user interfaces. It includes the MATLAB desktop and Command Window, a command history, an editor and debugger, and browsers for viewing help, the workspace, files, and the search path.

The MATLAB Mathematical Function Library. This is a vast collection of computational algorithms ranging from elementary functions, like sum, sine, cosine, and complex arithmetic, to more sophisticated functions like matrix inverse, matrix eigenvalues, Bessel functions, and fast Fourier transforms.

The MATLAB Language. This is a high-level matrix/array language with control flow statements, functions, data structures, input/output, and object-oriented programming features. It allows both "programming in the small" to rapidly create quick and dirty throw-away programs, and "programming in the large" to create large and complex application programs.

Graphics. MATLAB has extensive facilities for displaying vectors and matrices as graphs, as well as annotating and printing these graphs. It includes high-level functions for two-dimensional and three-dimensional data visualization, image processing, animation, and presentation graphics. It also includes low-level functions that allow you to fully customize the appearance of graphics as well as to build complete graphical user interfaces on your MATLAB applications.

The MATLAB External Interfaces (API). This is a library that allows you to write C and Fortran programs that interact with MATLAB. It includes facilities for calling routines from MATLAB (dynamic linking), calling MATLAB as a computational engine, and for reading and writing MAT-files.



What Is MATLAB?

MATLAB® is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical uses include Math and computation Algorithm development Data acquisition Modeling, simulation, and prototyping Data analysis, exploration, and visualization Scientific and engineering graphics Application development, including graphical user interface building

MATLAB is an interactive system whose basic data element is an array that does not require dimensioning. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a scalar noninteractive language such as C or Fortran.

The name MATLAB stands for matrix laboratory. MATLAB was originally written to provide easy access to matrix software developed by the LINPACK and EISPACK projects. Today, MATLAB engines incorporate the LAPACK and BLAS libraries, embedding the state of the art in software for matrix computation.

What Is MATLAB?

MATLAB® is a high-performance language for technical computing. It integrates computation, visualization, and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical uses include Math and computation Algorithm development Data acquisition Modeling, simulation, and prototyping Data analysis, exploration, and visualization Scientific and engineering graphics Application development, including graphical user interface building

MATLAB is an interactive system whose basic data element is an array that does not require dimensioning. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a scalar noninteractive language such as C or Fortran.

The name MATLAB stands for matrix laboratory. MATLAB was originally written to provide easy access to matrix software developed by the LINPACK and EISPACK projects. Today, MATLAB engines incorporate the LAPACK and BLAS libraries, embedding the state of the art in software for matrix computation.

MATLAB has evolved over a period of years with input from many users. In university environments, it is the standard instructional tool for introductory and advanced courses in mathematics, engineering, and science. In industry, MATLAB is the tool of choice for high-productivity research, development, and analysis.

MATLAB features a family of add-on application-specific solutions called toolboxes. Very important to most users of MATLAB, toolboxes allow you to learn and apply specialized technology. Toolboxes are comprehensive collections of MATLAB functions (M-files) that extend the MATLAB environment to solve particular classes of problems. Areas in which toolboxes are available include signal processing, control systems, neural networks, fuzzy logic, wavelets, simulation, and many others.

The MATLAB System

The MATLAB system consists of five main parts:

Development Environment. This is the set of tools and facilities that help you use MATLAB functions and files. Many of these tools are graphical user interfaces. It includes the MATLAB desktop and Command Window, a command history, an editor and debugger, and browsers for viewing help, the workspace, files, and the search path.

The MATLAB Mathematical Function Library. This is a vast collection of computational algorithms ranging from elementary functions, like sum, sine, cosine, and complex arithmetic, to more sophisticated functions like matrix inverse, matrix eigenvalues, Bessel functions, and fast Fourier transforms.

The MATLAB Language. This is a high-level matrix/array language with control flow statements, functions, data structures, input/output, and object-oriented programming features. It allows both "programming in the small" to rapidly create quick and dirty throw-away programs, and "programming in the large" to create large and complex application programs.

Graphics. MATLAB has extensive facilities for displaying vectors and matrices as graphs, as well as annotating and printing these graphs. It includes high-level functions for two-dimensional and three-dimensional data visualization, image processing, animation, and presentation graphics. It also includes low-level functions that allow you to fully customize the appearance of graphics as well as to build complete graphical user interfaces on your MATLAB applications.

The MATLAB External Interfaces (API). This is a library that allows you to write C and Fortran programs that interact with MATLAB. It includes facilities for calling routines from MATLAB (dynamic linking), calling MATLAB as a computational engine, and for reading and writing MAT-files.

Pipeline complete para segmentação:

Imagem → Top-hat → Limiarização → Morfologia → Labeling